第五篇 【实践与代码篇】可运行算法原型库

本篇所有代码均逐字取自 code/ 目录下的真实文件，由 tools/build_whitepaper.py 自动嵌入。
换句话说：你在这里看到的代码，就是仓库里能跑起来的那个文件，不存在「文档里一份、仓库里另一份」的情况。

运行环境与依赖见 code/requirements.txt（仅 numpy）与 code/README.md。

code/benchmarks.py

```
"""benchmarks.py —— 优化算法测试函数集
本模块收集本全集中各篇论文反复使用的**解析基准函数**，用于在没有 CFD 的情况下
验证优化算法的正确性与相对优劣。
选这些函数的理由（都对应到具体论文）：
* Hartman6 / Shekel4 / Camel6 / Trid10 —— 论文 01（CR-EI）的 7 个基准函数中的 4 个
* Acklev / Rosenbrock（含 6 维版本）—— 论文 06（GMFoO）与论文 08（KT-ASO）的
源/目标任务构造直接使用了 Acklev6d 与 0.9・Acklev6d + 0.1・Rosenbrock6d
* Acklev5 / Hartman6 —— 论文 02（Filter-GEI）Table 4 的测试函数
* 30~90 维版本 —— 论文 11（DA-EGO）在 21 个 30~90 维函数上验证
* 50~100 维版本 —— 论文 07（GSDE）的数值验证区间
**重要提醒**：这些函数是"廉价黑箱"，与真实 CFD 的昂贵性无关；它们只能验证
"算法的搜索逻辑"，不能验证"算法在昂贵场景下的样本效率"。真要评估后者，必须
统计**函数评估次数**（本模块中每个 Problem 都会记录 eval_count）。
运行自检：
python code/benchmarks.py
自检内容：已知全局最优值、维度与边界一致性、评估计数、以及"越接近最优点函数值越小"的
单调性抽查。
"""
from __future__ import annotations
from typing import Callable, Dict, List, Optional, Tuple
import numpy as np
    all     = [
"Problem",
"get_problem",
"list_problems",
"sphere", "rosenbrock", "acklev", "griewank", "levy", "trid",
"hartman6", "shekel4", "branin", "camel6",
]
# ---------------------------------------------------------------------------- 容器
class Problem:
"""一个带边界、已知最优值与评估计数的优化问题（统一按**最小化**处理）。"""
def __init__(self,
name: str,
func: Callable[[np.ndarray], float],
dim: int,
bounds: List[Tuple[float, float]],
fmin: float,
xmin: Optional[np.ndarray] = None,
note: str = ""):
self.name = name
self.func = func
self.dim = dim
self.bounds = bounds
self.fmin = fmin # 已知全局最小值（文献值）
self.xmin = None if xmin is None else np.asarray(xmin, dtype=float)
self.note = note
self.eval_count: int = 0 # 昂贵评估次数统计
self.history: List[float] = [] # 每次评估的目标值（便于画收敛曲线）
def __call__(self, x: np.ndarray) -> float:
x = np.asarray(x, dtype=float).ravel()
if x.size != self.dim:
raise ValueError(f"{self.name} 期望 {self.dim} 维，收到 {x.size} 维")
y = float(self.func(x))
self.eval_count += 1
self.history.append(y)
return y
def reset(self) -> None:
"""清空评估计数与历史（做多次独立重复实验时用）。"""
self.eval_count = 0
self.history.clear()
def best_so_far(self) -> float:
"""当前已评估样本中的最优值（用于画收敛曲线）。"""
if not self.history:
return float("inf")
return float(np.min(self.history))
def regret(self) -> float:
"""当前最优值与已知全局最优的差（越小越好，0 表示已找到全局最优）。"""
return self.best_so_far() - self.fmin
def __repr__(self) -> str: # pragma: no cover - 仅用于调试输出
return f"Problem({self.name}, dim={self.dim}, f*={self.fmin:.6g}, n_eval={self.eval_count})"
# ---------------------------------------------------------------------------- 基本函数
def sphere(x: np.ndarray) -> float:
"""Sphere：最平凡的凸单峰函数，全局最优 0（原点）。"""
x = np.asarray(x, dtype=float).ravel()
return float(np.sum(x ** 2))
def rosenbrock(x: np.ndarray) -> float:
```

```
    """Rosenbrock：经典"香蕉谷"，全局最优 0（全 1 向量）。
    论文 06/08 用的是带平移的版本：f(x) = Σ(0.4 - x_i)² + 20·Σ(x_i - x_{i+1})²，
    即最优在 x_i = 0.4；本处提供标准版本，平移版本见 make_shifted_rosenbrock()。
    """
    x = np.asarray(x, dtype=float).ravel()
    return float(np.sum(100.0 * (x[1:] - x[:-1] ** 2) ** 2 + (1.0 - x[:-1]) ** 2))

def make_shifted_rosenbrock(dim: int = 6) -> Callable[[np.ndarray], float]:
    """论文 08（KT-ASO）数值验证中使用的平移 Rosenbrock：
    f(x) = Σ_{i=1}^{d}(0.4 - x_i)² + 20·Σ_{i=1}^{d-1}(x_i - x_{i+1})²
    定义域 x_i ∈ [-2, 2]，最优在 x_i = 0.4，最优值 0。
    """
    def f(x: np.ndarray) -> float:
        x = np.asarray(x, dtype=float).ravel()
        return float(np.sum((0.4 - x) ** 2) + 20.0 * np.sum((x[:-1] - x[1:]) ** 2))
    f.__name__ = f"shifted_rosenbrock{dim}d"
    return f

def ackley(x: np.ndarray, a: float = 20.0, b: float = 0.2, c: float = 2.0 * np.pi) -> float:
    """Ackley：海量局部极小 + 全局最优 0（原点）。高维优化的常用试金石。"""
    x = np.asarray(x, dtype=float).ravel()
    d = x.size
    s1 = np.sum(x ** 2)
    s2 = np.sum(np.cos(c * x))
    return float(-a * np.exp(-b * np.sqrt(s1 / d)) - np.exp(s2 / d) + a + np.e)

def griewank(x: np.ndarray) -> float:
    """Griewank：乘积型多峰，全局最优 0（原点）。"""
    x = np.asarray(x, dtype=float).ravel()
    d = x.size
    i = np.arange(1, d + 1, dtype=float)
    return float(1.0 + np.sum(x ** 2) / 4000.0 - np.prod(np.cos(x / np.sqrt(i))))

def levy(x: np.ndarray) -> float:
    """Levy：全局最优 0（全 1 向量）。"""
    x = np.asarray(x, dtype=float).ravel()
    w = 1.0 + (x - 1.0) / 4.0
    term1 = np.sin(np.pi * w[0]) ** 2
    term3 = (w[-1] - 1.0) ** 2 * (1.0 + np.sin(2.0 * np.pi * w[-1]) ** 2)
    term2 = np.sum((w[:-1] - 1.0) ** 2 * (1.0 + 10.0 * np.sin(np.pi * w[:-1] + 1.0) ** 2))
    return float(term1 + term2 + term3)

def trid(x: np.ndarray) -> float:
    """Trid：全局最优 -d(d+4)(d-1)/6（论文 01 用 Trid10，最优值 -210）。"""
    x = np.asarray(x, dtype=float).ravel()
    d = x.size
    s1 = np.sum((x - 1.0) ** 2)
    s2 = np.sum(x[1:] * x[:-1])
    return float(s1 - s2)

def branin(x: np.ndarray) -> float:
    """Branin（2 维）：三个全局最优点，最优值 0.397887。"""
    x = np.asarray(x, dtype=float).ravel()
    x1, x2 = x[0], x[1]
    a, b, c, r, s, t = 1.0, 5.1 / (4.0 * np.pi ** 2), 5.0 / np.pi, 6.0, 10.0, 1.0 / (8.0 * np.pi)
    return float(a * (x2 - b * x1 ** 2 + c * x1 - r) ** 2 + s * (1.0 - t) * np.cos(x1) + s)

def camel6(x: np.ndarray) -> float:
    """Six-Hump Camel（2 维）：六个局部极小，两个全局最优点，最优值 -1.0316。
    论文 01（CR-EI）用它做参数敏感性分析。"""
    x = np.asarray(x, dtype=float).ravel()
    x1, x2 = x[0], x[1]
    return float((4.0 - 2.1 * x1 ** 2 + x1 ** 4 / 3.0) * x1 ** 2
                 + x1 * x2 + (-4.0 + 4.0 * x2 ** 2) * x2 ** 2)

# Hartman6 标准参数（论文 01 引用）
_H6_ALPHA = np.array([1.0, 1.2, 3.0, 3.2])
_H6_A = np.array([[10.0, 3.0, 17.0, 3.5, 1.7, 8.0],
                  [0.05, 10.0, 17.0, 0.1, 8.0, 14.0],
                  [3.0, 3.5, 1.7, 10.0, 17.0, 8.0],
                  [17.0, 8.0, 0.05, 10.0, 0.1, 14.0]])
_H6_P = 1e-4 * np.array([[1312, 1696, 5569, 124, 8283, 5886],
                         [2329, 4135, 8307, 3736, 1004, 9991],
                         [2348, 1451, 3522, 2883, 3047, 6650],
                         [4047, 8828, 8732, 5743, 1091, 381]])

def hartman6(x: np.ndarray) -> float:
    """Hartman6（6 维，域 [0,1]^6）：6 个局部最优，全局最小 -3.32237。
    论文 01 中明确提到"Hartman6 有 6 个局部最优"。"""
    x = np.asarray(x, dtype=float).ravel()
    return float(-np.sum(_H6_ALPHA * np.exp(-np.sum(_H6_A * (x[None, :] - _H6_P) ** 2, axis=1))))

# Shekel 标准参数（m=10）
_S4_A = np.array([[4.0, 4.0, 4.0, 4.0],
                  [1.0, 1.0, 1.0, 1.0],
                  [8.0, 8.0, 8.0, 8.0],
                  [6.0, 6.0, 6.0, 6.0],
                  [3.0, 7.0, 3.0, 7.0],
                  [2.0, 9.0, 2.0, 9.0],
                  [5.0, 5.0, 3.0, 3.0],
                  [8.0, 1.0, 8.0, 1.0],
                  [6.0, 2.0, 6.0, 2.0],
                  [7.0, 3.6, 7.0, 3.6]], dtype=float)
_S4_C = np.array([0.1, 0.2, 0.2, 0.4, 0.4, 0.6, 0.3, 0.7, 0.5, 0.5])

def shekel4(x: np.ndarray) -> float:
    """Shekel / Shekel4（4 维，域 [0,10]^4）：论文 01 Table 1 使用的函数之一。
    这里采用 m=10 的常用 Shekel 形式，全局最小约 -10.5364。"""
    x = np.asarray(x, dtype=float).ravel()
    inner = np.sum((x[None, :] - _S4_A) ** 2, axis=1) + _S4_C
    return float(-np.sum(1.0 / inner))

# ---------------------------------------------------------------------- 注册表

def _reg() -> Dict[str, Problem]:
    p: Dict[str, Problem] = {}
```

```
def add(prob: Problem) -> None:
p[prob.name] = prob
add(Problem("sphere5", sphere, 5, [(-5.12, 5.12)] * 5, 0.0, np.zeros(5),
"凸单峰，用于 sanity check"))
add(Problem("branin2", branin, 2, [(-5.0, 10.0), (0.0, 15.0)], 0.397887,
np.array([-np.pi, 12.275]),
"2 维，3 个全局最优点"))
add(Problem("camel6", camel6, 2, [(-3.0, 3.0), (-2.0, 2.0)], -1.031628,
np.array([0.0898, -0.7126]),
"论文 01 参数敏感性算例"))
add(Problem("hartman6", hartman6, 6, [(0.0, 1.0)] * 6, -3.322368,
np.array([0.20169, 0.150011, 0.476874, 0.275332, 0.311652, 0.6573]),
"论文 01/02 均使用；6 个局部最优"))
add(Problem("shekel4", shekel4, 4, [(0.0, 10.0)] * 4, -10.5364,
np.array([4.0, 4.0, 4.0, 4.0]),
"论文 01 Table 1 使用"))
add(Problem("trid10", trid, 10, [(-100.0, 100.0)] * 10, -210.0,
np.arange(1, 11) * (11 - np.arange(1, 11)),
"论文 01 Table 1 使用；最优值 -d(d+4)(d-1)/6"))
add(Problem("acklev5", acklev, 5, [(-32.768, 32.768)] * 5, 0.0, np.zeros(5),
"论文 02 Table 4 使用"))
add(Problem("acklev6", acklev, 6, [(-2.0, 2.0)] * 6, 0.0, np.zeros(6),
"论文 06/08 数值验证的目标任务"))
add(Problem("rosenbrock6", make_shifted_rosenbrock(6), 6, [(-2.0, 2.0)] * 6, 0.0,
np.full(6, 0.4),
"论文 08 的平移 Rosenbrock（最优在 x_i=0.4）"))
add(Problem("griewank10", griewank, 10, [(-600.0, 600.0)] * 10, 0.0, np.zeros(10),
"乘积型多峰"))
add(Problem("levy10", levy, 10, [(-10.0, 10.0)] * 10, 0.0, np.ones(10),
"多峰，最优在全 1 向量"))
add(Problem("rosenbrock30", rosenbrock, 30, [(-2.048, 2.048)] * 30, 0.0, np.ones(30),
"论文 11 的 30 维量级"))
add(Problem("acklev50", acklev, 50, [(-32.768, 32.768)] * 50, 0.0, np.zeros(50),
"论文 07 的 50~100 维量级"))
add(Problem("sphere100", sphere, 100, [(-5.12, 5.12)] * 100, 0.0, np.zeros(100),
"论文 07 的 100 维量级"))
return p
_REGISTRY: Dict[str, Problem] = _reg()
def list_problems() -> List[str]:
"""返回所有可用基准问题的名字。"""
return sorted(_REGISTRY)
def get_problem(name: str) -> Problem:
"""按名字取一个**全新的** Problem 实例（每次调用都会重置评估计数）。"""
if name not in _REGISTRY:
raise KeyError(f"未知问题 {name!r}，可用：{list_problems()}")
src = _REGISTRY[name]
return Problem(src.name, src.func, src.dim, list(src.bounds), src.fmin, src.xmin, src.note)
# ---------------------------------------------------------------------------- 自检
def _self_test() -> None:
# 1) 已知函数的最优值核对（含论文 01 提到的 Trid10 = -210）
checks = [
("sphere5", np.zeros(5), 0.0),
("branin2", np.array([-np.pi, 12.275]), 0.397887),
("camel6", np.array([0.0898, -0.7126]), -1.0316),
("hartman6", np.array([0.20169, 0.150011, 0.476874, 0.275332, 0.311652, 0.6573]), -3.32237),
("trid10", np.arange(1, 11) * (11 - np.arange(1, 11)), -210.0),
("acklev5", np.zeros(5), 0.0),
("acklev6", np.zeros(6), 0.0),
("rosenbrock6", np.full(6, 0.4), 0.0),
("griewank10", np.zeros(10), 0.0),
("levy10", np.ones(10), 0.0),
]
for name, xstar, fstar in checks:
pr = get_problem(name)
got = pr(xstar)
assert abs(got - fstar) < 5e-3, f"{name} 最优值不符：{got} vs {fstar}"
assert got <= pr.fmin + 5e-3, f"{name} 的 fmin 登记值偏小"
# 2) 维度与边界一致
for name in list_problems():
pr = get_problem(name)
assert len(pr.bounds) == pr.dim, f"{name} 边界数与维度不一致"
for lo, hi in pr.bounds:
assert hi > lo, f"{name} 存在非法边界"
# 3) 评估计数与收敛记录
pr = get_problem("hartman6")
rng = np.random.default_rng(7)
for _ in range(10):
pr(rng.random(6))
assert pr.eval_count == 10, "评估计数错误"
assert len(pr.history) == 10, "历史长度错误"
assert pr.best_so_far() == min(pr.history), "best_so_far 计算错误"
pr.reset()
assert pr.eval_count == 0 and not pr.history, "reset 未清空"
# 4) 维度错误应报错
try:
pr(rng.random(5))
except ValueError:
pass
else:
raise AssertionError("维度不匹配未报错")
# 5) 单调性抽查：沿"当前点->全局最优点"连线插值，函数值应大体下降
pr = get_problem("ackley5")
```

```
lo = np.array([b[0] for b in pr.bounds])
hi = np.array([b[1] for b in pr.bounds])
start = lo + 0.25 * (hi - lo)
vals = [pr(start + t * (pr.xmin - start)) for t in np.linspace(0, 1, 9)]
assert vals[-1] < vals[0], "沿指向最优点的路径函数值未下降"
print(f"[benchmarks.py] 自检通过：{len(list_problems())} 个问题，"
f"最优值/边界/计数/单调性 全部正确")
if __name__ == "__main__":
_self_test()
```

code/decomposition.py

```
"""decomposition.py —— 高维分解：变量交互检测、子空间分组、精英点聚合（DA-EGO）
第 04 讲（**论文 11，DA-EGO**）要解决的是第 03 讲（论文 07，GSDE）留下的另一个方向：
GSDE 选择"放弃贝叶斯、拥抱进化"，DA-EGO 则反问"能不能改造贝叶斯框架本身来扛高维"。
它的答案是 **动态分解与聚合**：
[知识] → 构造子任务（分解方案 + 搜索范围）
↓
各子空间独立做基于代理的优化
↓
子任务数据挖掘（摄动法 + ANOVA 检测交互）
↓
[知识更新] → 下一轮的分解方案与搜索范围
其中的三个关键构件，本模块都给出了可运行实现：
| 函数 | 对应论文 11 的环节 |
| :--- | :--- |
| `interaction_scores` | 变量交互强度打分（对应原文的 PCE 灵敏度系数 SPCE_ij） |
| `select_interaction_pairs` | 把变量对分成 **确认交互 P_C / 疑似交互 P_S / 非交互** |
| `group_variables` | 据已确认的交互对把 d 维空间切成若干低维子空间 |
| `combine_into_elite` | **精英点（elite-point）**：把各子空间的最优解拼回全空间 |
| `shrink_bounds` | 依据历史精英点收缩搜索范围 |
> ⚠ **两处必须说清楚的实现差异**：
> 1. 原文的交互打分用**多项式混沌展开（PCE）的灵敏度系数 SPCE_ij**；本模块用的是
> **在代理模型上做蒙特卡洛 ANOVA 的二阶交互效应**。PCE 需要专门的基函数与系数
> 求解（且原文自己都承认"高维 PCE 代理精度有限，会导致交互变量对选取出现误差"），
> 本实现选择了一个同样基于方差分解、但更省事且行为可预期的替代方案。
> 2. 原文 P_S 中的疑似交互还要**再用低维代理模型验证确认**；本模块把这一步留成
> `confirm_pairs` 的回调接口（可用 `code/gp.py` 的 Kriging 实现）。
**只依赖 numpy**。运行自检：
python code/decomposition.py
"""
from __future__ import annotations
from typing import Callable, Dict, Iterable, List, Optional, Sequence, Tuple
import numpy as np
__all__ = [
"interaction_scores", "select_interaction_pairs", "group_variables",
"combine_into_elite", "shrink_bounds", "subspace_bounds",
]
_TINY = 1e-12
# ---------------------------------------------------------------------------- 交互强度
def interaction_scores(model: Callable[[np.ndarray], np.ndarray],
bounds: Sequence[Tuple[float, float]],
pairs: Optional[Iterable[Tuple[int, int]]] = None,
n_mc: int = 1024,
seed: int = 0) -> Dict[Tuple[int, int], float]:
"""估计每一对变量的**二阶交互效应**（归一化到总方差，越大越该放同一子空间）。
采用 pick-freeze（Sobol 经典套路）在**代理模型**上做蒙特卡洛：给定两张独立的
随机矩阵 A、B，记 A^{i←B} 为把 A 的第 i 列换成 B 的第 i 列，则"纯交互"部分可用
混合二阶差分刻画：
Δ_ij = f(A) − f(A^{i←B}) − f(A^{j←B}) + f(A^{ij←B})
S_ij^int = E[Δ_ij²] / (4 · Var(f))
若 x_i 与 x_j 可加分离（无交互），Δ_ij ≡ 0，故该分数为 0。
成本提示：每个变量对需要 4 次长度为 n_mc 的代理评估；d=126 时共 7875 对
（126×125/2），即约 3 万次批量评估。这正是为什么必须先用**便宜的代理**而不是 CFD。
"""
bounds = np.asarray(bounds, float)
d = bounds.shape[0]
lo, span = bounds[:, 0], bounds[:, 1] - bounds[:, 0]
rng = np.random.default_rng(seed)
if pairs is None:
pairs = [(i, j) for i in range(d) for j in range(i + 1, d)]
pairs = [tuple(sorted(p)) for p in pairs]
for i, j in pairs:
if not (0 <= i < d and 0 <= j < d) or i == j:
raise ValueError(f"非法的变量对：({i}, {j})")
A = lo + rng.random((n_mc, d)) * span
B = lo + rng.random((n_mc, d)) * span
def f(X: np.ndarray) -> np.ndarray:
return np.asarray(model(X), float).ravel()
fA = f(A)
tot = float(np.var(np.concatenate([fA, f(B)])))
if tot <= _TINY:
return {p: 0.0 for p in pairs}
cache: Dict[int, np.ndarray] = {}
def col_swap(src: np.ndarray, k: int) -> np.ndarray:
"""把 src 的第 k 列换成 B 的第 k 列。"""
key = (id(src), k)
if key in cache:
```

```
    return cache[key]
    out = src.copy()
    out[:, k] = B[:, k]
    cache[key] = out
    return out

scores: Dict[Tuple[int, int], float] = {}
for (i, j) in pairs:
    Ai = col_swap(A, i)
    Aj = col_swap(A, j)
    Aij = col_swap(Ai, j)
    delta = fA - f(Ai) - f(Aj) + f(Aij)
    scores[(i, j)] = float(np.mean(delta ** 2) / (4.0 * tot))
return scores

def select_interaction_pairs(scores: Dict[Tuple[int, int], float],
                             confirmed: Optional[Iterable[Tuple[int, int]]] = None,
                             n_suspected: int = 10,
                             threshold: float = 0.01) -> Dict[str, List[Tuple[int, int]]]:
    """把变量对分成三类［原文 P11 §3.2.1］：

    * `confirmed`（P_C）：已确认有交互的变量对
    * `suspected`（P_S）：在**剩余**变量对里分数最高的一批，待验证
    * `ignored`：其余（认为交互很弱，可以安全地分进不同子空间）

    threshold 是"分数低于此值就认为无交互"的门槛；n_suspected 控制 P_S 的规模。
    """
    confirmed = [tuple(sorted(c)) for c in (confirmed or [])]
    rest = {p: s for p, s in scores.items()
            if p not in set(confirmed) and s >= threshold}
    ranked = sorted(rest.items(), key=lambda kv: -kv[1])
    suspected = [p for p, _ in ranked[:max(0, int(n_suspected))]]
    suspected_set = set(suspected)
    return {
        "confirmed": confirmed,
        "suspected": suspected,
        "ignored": [p for p in scores if p not in set(confirmed) and p not in suspected_set],
    }

# ---------------------------------------------------------------------- 子空间分组

def group_variables(d: int, pairs: Iterable[Tuple[int, int]]) -> List[List[int]]:
    """据交互对做并查集分组：有交互的变量必须落进**同一个**子空间，
    相互独立的变量单独成组。返回若干组变量下标。"""
    parent = list(range(d))

    def find(a: int) -> int:
        while parent[a] != a:
            parent[a] = parent[parent[a]]
            a = parent[a]
        return a

    def union(a: int, b: int) -> None:
        ra, rb = find(a), find(b)
        if ra != rb:
            parent[max(ra, rb)] = min(ra, rb)

    for i, j in pairs:
        if not (0 <= i < d and 0 <= j < d):
            raise ValueError(f"非法的变量对：({i}, {j})")
        union(int(i), int(j))

    groups: Dict[int, List[int]] = {}
    for v in range(d):
        groups.setdefault(find(v), []).append(v)
    return [sorted(g) for _, g in sorted(groups.items(), key=lambda kv: kv[1][0])]

def subspace_bounds(bounds: Sequence[Tuple[float, float]],
                    groups: Sequence[Sequence[int]]) -> List[np.ndarray]:
    """取出每个子空间对应的变量边界，便于在各子空间里独立跑 EGO。"""
    b = np.asarray(bounds, float)
    out = []
    for g in groups:
        idx = list(g)
        out.append(b[idx].copy())
    return out

# ---------------------------------------------------------------------- 精英点聚合

def combine_into_elite(elite: np.ndarray,
                       solutions: Sequence[Tuple[Sequence[int], np.ndarray]]) -> np.ndarray:
    """精英点（elite-point）聚合［原文 P11 §3］。

    `solutions` 是一串 (变量下标, 该子空间的最优解) 对；函数把每个子空间的最优片段
    **写回精英点对应的维度**，得到一个全空间的完整可行解。

    这一步的意义：子空间是低维的、好优化的，但只有"局部视野"；精英点把各局部视野的
    最优解**拼装成一个全局可行解**，保证子空间优化不会跑偏到全空间的劣解上。
    """
    out = np.array(elite, dtype=float).copy()
    for idx, vals in solutions:
        idx = list(idx)
        vals = np.asarray(vals, float).ravel()
        if len(idx) != vals.size:
            raise ValueError("子空间变量下标数与解的长度不一致")
        if np.max(idx) >= out.size:
            raise ValueError("子空间下标越界")
        out[idx] = vals
    return out

# ---------------------------------------------------------------------- 搜索范围收缩

def shrink_bounds(bounds: Sequence[Tuple[float, float]],
                  elite_history: Sequence[np.ndarray],
                  rate: float = 0.5,
                  keep_center_last: bool = True) -> np.ndarray:
    """依据历史精英点收缩搜索范围［原文 P11 §3.2.2］。

    若此前求解过**相似子任务**（变量相同、精英点不同），可依据累积经验缩小搜索范围，
```

```
从而同时提升优化效率与交互分析的精度。
做法：以历史精英点的取值范围为基准，按 `rate` 向中心收缩（rate=0.5 表示新区间
宽度是历史精英点范围的一半），并与原边界取交集，保证不越界。
"""
b = np.asarray(bounds, float).copy()
H = np.atleast_2d(np.asarray(elite_history, float))
if H.size == 0 or H.shape[0] < 2:
return b
if not (0.0 < rate <= 1.0):
raise ValueError("rate 应落在 (0, 1]")
lo_h, hi_h = H.min(axis=0), H.max(axis=0)
center = 0.5 * (lo_h + hi_h) if not keep_center_last else H[-1]
half = 0.5 * rate * (hi_h - lo_h)
new_lo = np.maximum(b[:, 0], center - half)
new_hi = np.minimum(b[:, 1], center + half)
bad = new_hi <= new_lo
new_lo[bad], new_hi[bad] = b[bad, 0], b[bad, 1]
return np.column_stack([new_lo, new_hi])
# ---------------------------------------------------------------------------- 自检
def _self_test() -> None:
rng = np.random.default_rng(20240905)
d = 5
bounds = [(0.0, 1.0)] * d
# 真值函数：(0,1) 强交互；(2,3) 弱交互；其余项可加分离（交互应恰为 0）
def truth(X: np.ndarray) -> np.ndarray:
X = np.atleast_2d(np.asarray(X, float))
strong = 5.0 * X[:, 0] * X[:, 1] # 强交互
weak = 0.3 * X[:, 2] * X[:, 3] # 弱交互
additive = 1.0 * X[:, 2] + 0.8 * X[:, 3] ** 2 # 可加项
tiny = 0.05 * X[:, 4] # 几乎无影响
return strong + weak + additive + tiny
scores = interaction_scores(truth, bounds, n_mc=4096, seed=3)
assert len(scores) == d * (d - 1) // 2, "变量对数量不对"
pairs_sorted = sorted(scores.items(), key=lambda kv: -kv[1])
top = pairs_sorted[0][0]
assert top == (0, 1), f"最强交互对应被误判：{top}"
assert scores[(0, 1)] > 5 * scores[(0, 2)], "交互项未显著高于无关变量对"
# 可加部分（1·x2 + 0.8·x3²）之间不应产生交互，估计值应恰为 0
assert scores[(0, 4)] < 1e-12, f"可加项被误判为有交互：{scores[(0, 4)]}"
assert scores[(1, 4)] < 1e-12, f"可加项被误判为有交互：{scores[(1, 4)]}"
# 弱交互应被检出，但量级远小于强交互
assert 0 < scores[(2, 3)] < 0.05 * scores[(0, 1)], \
f"弱交互量级不对：{scores[(2, 3)]}"
assert all(v >= 0 for v in scores.values()), "交互分数不应为负"
# 分类：确认对 + 疑似对 + 其余（三者必须构成全集的划分）
sel = select_interaction_pairs(scores, confirmed=[(0, 1)],
n_suspected=3, threshold=1e-4)
assert (0, 1) in sel["confirmed"], "已确认对丢失"
assert len(sel["suspected"]) <= 3 and len(sel["suspected"]) >= 1, "疑似对数量不对"
assert (0, 1) not in sel["suspected"], "已确认对不应再进疑似集"
assert (2, 3) in sel["suspected"], "弱交互对应进入疑似集待验证"
assert len(sel["confirmed"]) + len(sel["suspected"]) + len(sel["ignored"]) == len(scores)
assert not (set(sel["confirmed"]) & set(sel["suspected"])), "三类划分存在重叠"
# 分组：有交互的必须同组，其余各自成组
groups = group_variables(d, [(0, 1)])
assert [0, 1] in groups, f"(0,1) 未分到同组：{groups}"
assert len(groups) == 4, f"应分成 4 组，实为 {len(groups)}：{groups}"
assert sorted(sum(groups, [])) == list(range(d)), "分组未覆盖全部变量"
# 传递性：(0,1) 与 (1,4) → {0,1,4} 必须同组
g2 = group_variables(d, [(0, 1), (1, 4)])
assert [0, 1, 4] in g2, f"并查集未体现传递性：{g2}"
# 子空间边界
sb = subspace_bounds(bounds, groups)
assert len(sb) == len(groups), "子空间边界数量不对"
assert sb[0].shape == (2, 2), "含 2 个变量的子空间应有 2 条边界"
# 精英点聚合：把两个子空间的最优片段写回
elite = np.zeros(d)
new_elite = combine_into_elite(elite, [([0, 1], np.array([0.9, 0.8])),
([2], np.array([0.3]))])
assert np.allclose(new_elite, [0.9, 0.8, 0.3, 0.0, 0.0]), "精英点聚合错误"
assert np.allclose(elite, np.zeros(d)), "聚合不应修改原精英点（应返回新数组）"
# 搜索范围收缩：新区间必须落在原区间内，且不宽于原区间
hist = [np.array([0.4, 0.4, 0.4, 0.4, 0.4]),
np.array([0.6, 0.6, 0.6, 0.6, 0.6])]
nb = shrink_bounds(bounds, hist, rate=0.5)
assert np.all(nb[:, 0] >= 0.0) and np.all(nb[:, 1] <= 1.0), "收缩后越界"
assert np.all((nb[:, 1] - nb[:, 0]) <= 0.5 + 1e-12), "收缩幅度不足"
assert np.all((nb[:, 1] - nb[:, 0]) > 0), "收缩后区间退化"
# 只有一个历史精英点时不收缩
assert np.allclose(shrink_bounds(bounds, hist[:1]), np.asarray(bounds, float))
# 端到端：在这个"只有一对交互"的函数上，分解后应识别出 1 个 2 维子空间 + 3 个 1 维
sel2 = select_interaction_pairs(scores, n_suspected=8, threshold=0.005)
confirmed = [p for p in sel2["suspected"] if scores[p] > 0.2 * scores[(0, 1)]]
g3 = group_variables(d, confirmed)
assert [0, 1] in g3, f"端到端分解未把 (0,1) 归入同组：{g3}"
# 异常输入
for bad in [lambda: interaction_scores(truth, bounds, pairs=[(0, 0)]),
lambda: interaction_scores(truth, bounds, pairs=[(0, 9)]),
lambda: group_variables(d, [(0, 7)]),
lambda: combine_into_elite(np.zeros(d), [([0, 1], np.array([1.0]))]),
lambda: combine_into_elite(np.zeros(d), [([9], np.array([1.0]))]),
lambda: shrink_bounds(bounds, hist, rate=1.5)]:
```

```
try:
bad()
except ValueError:
pass
else:
raise AssertionError("非法输入未报错")
print(f"[decomposition.py] 自检通过：交互检测(top={top}, S={scores[(0, 1)]:.3f}) / "
f"三类划分 / 并查集分组{g3} / 精英点聚合 / 边界收缩")
if __name__ == "__main__":
_self_test()
```

code/gp.py

```
"""gp.py —— 普通 Kriging（高斯过程回归）与 EGO 采集函数
本全集中以 **Kriging / 高斯过程** 为方法核心或基础组件的论文（均可在 corpus/ 中核对）：
* 论文 01（CR-EI）、论文 02（Filter-GEI）—— 在 Kriging 上构造期望改善类采集函数「原文 P01 §2 / P02 §2」
* 论文 03 —— 端壁造型三项指标各建 Kriging 代理，再用 EGO 分别优化「原文 P03 §2.4」
* 论文 04 —— 基于 Kriging 代理的高效 UO + 全局灵敏度分析「出版商页面 摘要，论文 04」
* 论文 06（GMFoO）、论文 08（KT-ASO）—— 贝叶斯优化内核以 GP 为代理「原文 P06 §III / P08 §3」
* 论文 17（GTO）—— 对比"co-kriging 型 MFS"与 EMFS 两种多保真代理「原文 P17 §4」
* 论文 16 —— 仅把 GPR 作为与 TNO 对比的 5 种传统代理之一「原文 P16 Table 7」
本模块给出"单保真度普通 Kriging"的最小可用实现：各向异性高斯相关函数、ARD 长度尺度
的集中似然估计、预测均值/方差，以及 EI / GEI 形式（论文 02）的采集函数。
（本模块不实现任何论文专有变体；论文 05 用的是均匀设计 + 方差分解而非 Kriging，见 sampling.py。）
数学形式（普通 Kriging 的标准写法）：
Y(x) = μ + Z(x), Corr[Z(x), Z(x')] = exp( - Σ_i θ_i (x_i - x'_i)² )
超参数 (μ, σ², θ) 由**集中对数似然**最大化得到：
ln L = -n/2 · ln σ² - 1/2 · ln|R|
其中 σ² = (y-1μ̂)' R⁻¹ (y-1μ̂)/n，μ̂ = (1'R⁻¹y)/(1'R⁻¹1)。
**只依赖 numpy**：超参数搜索用"坐标方向黄金分割 + 随机重启"实现（不引入 scipy）。
运行自检：
python code/gp.py
自检内容：训练点插值性质、留一误差量级、超参数恢复、EI 在已知最优处为 0、
预测方差 ≥0、以及 GP 代 ANOVA 一阶效应（论文 04 思路）的排序正确性。
"""
from __future__ import annotations
import math
from typing import Callable, List, Optional, Sequence, Tuple
import numpy as np
__all__ = [
"Kriging",
"squared_exponential_corr",
"expected_improvement",
"generalized_expected_improvement",
"first_order_effects",
"anova_variance_shares",
]
_TINY = 1e-12
_ERF = np.vectorize(math.erf, otypes=[float])
# ---------------------------------------------------------------------- 相关函数
def squared_exponential_corr(X: np.ndarray, Y: np.ndarray, theta: np.ndarray) -> np.ndarray:
"""各向异性（ARD）高斯相关矩阵：R[i,j] = exp(-Σ_k θ_k (X_ik - Y_jk)²)。
X: (n, d)，Y: (m, d)，theta: (d,)。X 与 Y 应已按同一尺度归一化。
"""
X = np.asarray(X, dtype=float)
Y = np.asarray(Y, dtype=float)
theta = np.asarray(theta, dtype=float).ravel()
if X.ndim == 1:
X = X.reshape(1, -1)
if Y.ndim == 1:
Y = Y.reshape(1, -1)
if X.shape[1] != Y.shape[1]:
raise ValueError("X 与 Y 的维度不一致")
if theta.size == 1 and X.shape[1] > 1:
theta = np.repeat(theta, X.shape[1])
# (n,1,d) - (1,m,d) -> (n,m,d) -> 加权平方 -> (n,m)
diff = X[:, None, :] - Y[None, :, :]
d2 = np.einsum("nmk,k->nm", diff ** 2, theta)
return np.exp(-d2)
def _chol_solve(L: np.ndarray, b: np.ndarray) -> np.ndarray:
"""用下三角 Cholesky 因子 L 解 (L Lᵀ) z = b。"""
y = np.linalg.solve(L, b)
return np.linalg.solve(L.T, y)
def _stable_chol(R: np.ndarray) -> Optional[np.ndarray]:
"""尝试 Cholesky 分解；失败返回 None（表明 θ 过大/样本重复导致矩阵接近奇异）。"""
try:
return np.linalg.cholesky(R)
except np.linalg.LinAlgError:
return None
# ---------------------------------------------------------------------- 主类
class Kriging:
"""普通 Kriging（常数趋势 + 高斯相关 + 小 nugget）。
典型用法：
gp = Kriging().fit(X, y) # X: (n,d) 原始尺度；y: (n,)
mu, sd = gp.predict(Xs) # 返回均值与标准差
loocv = gp.leave_one_out_rmse() # 模型可信度自检
"""
def __init__(self,
nugget: float = 1e-8,
```

```
theta0: Optional[np.ndarray] = None,
n_restarts: int = 3,
seed: int = 0):
self.nugget = float(nugget)
self.theta0 = None if theta0 is None else np.asarray(theta0, float).ravel()
self.n_restarts = int(n_restarts)
self.seed = int(seed)
# 拟合后写入
self.X: Optional[np.ndarray] = None # 归一化后的训练点 (n,d)
self.y: Optional[np.ndarray] = None # **标准化后**的响应（零均值单位方差）
self.y_mean: float = 0.0 # 标准化用的均值
self.y_std: float = 1.0 # 标准化用的标准差
self.theta: Optional[np.ndarray] = None # ARD 长度尺度参数（归一化尺度）
self.mu: float = 0.0 # 常数趋势 μ̂
self.sigma2: float = 1.0 # 过程方差 σ̂²
self.xmin: Optional[np.ndarray] = None # 原始尺度下界
self.xspan: Optional[np.ndarray] = None # 原始尺度跨度
self.Rinv_y: Optional[np.ndarray] = None
self.loglik: float = -np.inf
# ---------- 归一化工具
def _to_unit(self, A: np.ndarray) -> np.ndarray:
return (np.asarray(A, float) - self.xmin) / self.xspan
def _from_unit(self, A: np.ndarray) -> np.ndarray:
return np.asarray(A, float) * self.xspan + self.xmin
# ---------- 集中似然
def _loglik(self, theta: np.ndarray) -> Tuple[float, Optional[dict]]:
"""给定 θ 计算集中对数似然；同时返回中间量，避免重复分解。"""
R = squared_exponential_corr(self.X, self.X, theta)
R[np.diag_indices_from(R)] += self.nugget
L = _stable_chol(R)
if L is None:
return -np.inf, None
one = np.ones(self.X.shape[0])
Rinv_one = _chol_solve(L, one)
Rinv_y = _chol_solve(L, self.y)
mu = float(one @ Rinv_y) / float(one @ Rinv_one)
resid = self.y - one * mu
sigma2 = float(resid @ _chol_solve(L, resid)) / self.X.shape[0]
if sigma2 <= 0:
return -np.inf, None
logdet = 2.0 * float(np.sum(np.log(np.diag(L))))
ll = -0.5 * self.X.shape[0] * math.log(sigma2) - 0.5 * logdet
return ll, {"L": L, "mu": mu, "sigma2": sigma2, "Rinv_y": Rinv_y,
"Rinv_one": Rinv_one, "theta": np.asarray(theta, float).copy()}
def _search_theta(self, d: int) -> np.ndarray:
"""坐标方向黄金分割 + 随机重启搜索 log10(θ) ∈ [-3, 2]。"""
rng = np.random.default_rng(self.seed)
def golden(f: Callable[[float], float], a: float, b: float, iters: int = 18) -> float:
gr = (math.sqrt(5.0) - 1.0) / 2.0
c, dd = b - gr * (b - a), a + gr * (b - a)
fc, fd = f(c), f(dd)
for _ in range(iters):
if fc > fd:
b, dd, fd = dd, c, fc
c = b - gr * (b - a)
fc = f(c)
else:
a, c, fc = c, dd, fd
dd = a + gr * (b - a)
fd = f(dd)
return (a + b) / 2.0
starts: List[np.ndarray] = []
if self.theta0 is not None and self.theta0.size == d:
starts.append(np.log10(np.clip(self.theta0, 1e-3, 1e2)))
starts.append(np.zeros(d)) # θ = 1
starts.append(np.full(d, -1.0)) # θ = 0.1（平滑）
for _ in range(max(0, self.n_restarts - len(starts))):
starts.append(rng.uniform(-2.5, 1.5, size=d))
best_ll, best = -np.inf, np.zeros(d)
for s in starts:
cur = s.copy()
cur_ll, _ = self._loglik(10.0 ** cur)
if not np.isfinite(cur_ll):
cur_ll = -np.inf
for _ in range(3): # 3 轮坐标扫描
for k in range(d):
def f(v: float, k=k, cur=cur) -> float:
trial = cur.copy()
trial[k] = v
ll, _ = self._loglik(10.0 ** trial)
return -np.inf if not np.isfinite(ll) else ll
cur[k] = golden(f, -3.0, 2.0)
cur_ll, _ = self._loglik(10.0 ** cur)
if np.isfinite(cur_ll) and cur_ll > best_ll:
best_ll, best = cur_ll, cur.copy()
return 10.0 ** best
# ---------- 拟合
def fit(self, X: np.ndarray, y: np.ndarray) -> "Kriging":
X = np.asarray(X, dtype=float)
y = np.asarray(y, dtype=float).ravel()
if X.ndim == 1:
X = X.reshape(-1, 1)
```

```
if X.shape[0] != y.shape[0]:
raise ValueError("X 与 y 的样本数不一致")
if X.shape[0] < 2:
raise ValueError("至少需要 2 个样本")
self.xmin = X.min(axis=0)
self.xspan = X.max(axis=0) - self.xmin
self.xspan[self.xspan <= 0] = 1.0 # 常数列保护
# 输入归一化到 [0,1]^d，输出标准化到零均值单位方差
# ——两步都只为数值稳定：Kriging 的相关矩阵在原始量纲下条件数可能高达 1e8 以上，
# 会使"训练点处精确插值"这一理论性质在浮点运算中丢失。
self.X = self._to_unit(X)
self.y_mean = float(np.mean(y))
self.y_std = float(np.std(y))
self.y_std = self.y_std if self.y_std > _TINY else 1.0
self.y = (y - self.y_mean) / self.y_std
self.theta = self._search_theta(X.shape[1])
ll, aux = self._loglik(self.theta)
if aux is None: # 极端退化时退回平滑先验
self.theta = np.full(X.shape[1], 0.1)
ll, aux = self._loglik(self.theta)
if aux is None: # pragma: no cover
raise RuntimeError("Kriging 拟合失败：相关矩阵奇异")
self.mu = aux["mu"]
self.sigma2 = aux["sigma2"]
self.Rinv_y = aux["Rinv_y"]
self.Rinv_one = aux["Rinv_one"]
self.L = aux["L"]
self.loglik = ll
return self
# ---------- 预测
def predict(self, Xs: np.ndarray, return_var: bool = False):
"""返回 (均值, 标准差)；return_var=True 时第二项改为方差。"""
Xs = np.asarray(Xs, dtype=float)
if Xs.ndim == 1:
Xs = Xs.reshape(1, -1)
Zs = self._to_unit(Xs)
n = self.X.shape[0]
mu_list, s2_list = [], []
for z in Zs: # 逐点（n 通常不大，代码保持直白）
r = squared_exponential_corr(z.reshape(1, -1), self.X, self.theta).ravel()
Rinv_r = _chol_solve(self.L, r)
mu = self.mu + float(r @ (self.Rinv_y - self.mu * self.Rinv_one))
s2 = self.sigma2 * max(0.0, 1.0 + self.nugget - float(r @ Rinv_r))
mu_list.append(mu)
s2_list.append(max(s2, 0.0))
mean = np.array(mu_list) * self.y_std + self.y_mean
var = np.array(s2_list) * (self.y_std ** 2)
return (mean, var) if return_var else (mean, np.sqrt(var))
# ---------- 诊断
def leave_one_out_rmse(self) -> float:
"""留一交叉验证 RMSE（衡量代理模型可信度；论文 03/14 的模型验证思路）。"""
n = self.X.shape[0]
errs = []
for i in range(n):
keep = np.ones(n, dtype=bool)
keep[i] = False
sub = Kriging(nugget=self.nugget, theta0=self.theta,
n_restarts=1, seed=self.seed).fit(self.X[keep], self.y[keep])
# 子模型直接在"标准化坐标"里拟合与预测，故误差也是标准化单位
mu, _ = sub.predict(self.X[i].reshape(1, -1), return_var=False)
errs.append(float(mu[0]) - float(self.y[i]))
# 换算回原始量纲
return float(np.sqrt(np.mean(np.square(errs))) * self.y_std)
def r2(self, X: np.ndarray, y: np.ndarray) -> float:
"""在独立测试集上的决定系数 R²。"""
mu, _ = self.predict(X)
y = np.asarray(y, float).ravel()
ss_res = float(np.sum((y - mu) ** 2))
ss_tot = float(np.sum((y - np.mean(y)) ** 2))
if ss_tot <= _TINY:
return 1.0 if ss_res <= _TINY else 0.0
return 1.0 - ss_res / ss_tot
# ---------- 采集函数
def expected_improvement(mean: np.ndarray, sd: np.ndarray, f_best: float,
xi: float = 0.0) -> np.ndarray:
"""经典 EI（论文 01 的 CR-EI 即在此基础上替换"当前最优"的定义）。
EI(x) = (f_best - ξ - μ)·Φ(u) + s·φ(u)，u = (f_best - ξ - μ)/s
其中 φ、Φ 为标准正态密度与分布函数（用 math.erf 实现，不依赖 scipy）。
"""
mean = np.asarray(mean, float)
sd = np.asarray(sd, float)
imp = f_best - xi - mean
out = np.zeros_like(mean)
pos = sd > _TINY
u = np.zeros_like(mean)
u[pos] = imp[pos] / sd[pos]
phi = np.exp(-0.5 * u ** 2) / math.sqrt(2.0 * math.pi)
Phi = 0.5 * (1.0 + _ERF(u / math.sqrt(2.0)))
out[pos] = imp[pos] * Phi[pos] + sd[pos] * phi[pos]
return out
def generalized_expected_improvement(mean: np.ndarray, sd: np.ndarray,
f_best: float, g: int = 1) -> np.ndarray:
"""广义期望改善 GEI（论文 02 的 Filter-GEI 所用准则）：
```

```
GEI  g(x) = s^g · Σ_{k=0}^{g} (-1)^k · C(g,k) · u^{g-k} · T_k
其中 u = (f_best - μ)/s，T_k 由递推 T_0 = Φ(u)、T_1 = -φ(u)、
T_k = -u^{k-1} φ(u) + (k-1) T_{k-2} 给出。g=1 时退化为经典 EI。
"""
mean = np.asarray(mean, float)
sd = np.asarray(sd, float)
u = np.where(sd > _TINY, (f_best - mean) / np.where(sd > _TINY, sd, 1.0), 0.0)
phi = np.exp(-0.5 * u ** 2) / math.sqrt(2.0 * math.pi)
Phi = 0.5 * (1.0 + _ERF(u / math.sqrt(2.0)))
T = [Phi, -phi]
for k in range(2, g + 1):
T.append(-(u ** (k - 1)) * phi + (k - 1) * T[k - 2])
out = np.zeros_like(mean)
for k in range(g + 1):
coef = math.comb(g, k) * ((-1.0) ** k)
out += coef * (u ** (g - k)) * T[k]
return (sd ** g) * out
# ---------------------------------------------------------------------- 方差分解（论文 04/05）
def first_order_effects(model: Callable[[np.ndarray], np.ndarray],
bounds: Sequence[Tuple[float, float]],
n_mc: int = 4096,
seed: int = 0) -> Tuple[np.ndarray, np.ndarray]:
"""用代理模型（如拟合好的 Kriging）做一阶效应分析，对应论文 04 的 Sobol 敏感性。
采用 Saltelli 式两矩阵（A、B 与混合矩阵）的 Jansen 估计器：
S_i ≈ (1/(2N)) Σ (f(B) - f(B^i))^2 / Var(f)
其中 B^i 是 B 的第 i 列换成 A 的第 i 列。返回 (一阶指数, 各变量"主效应曲线"的方差)。
"""
rng = np.random.default_rng(seed)
bounds = np.asarray(bounds, float)
d = bounds.shape[0]
lo, span = bounds[:, 0], bounds[:, 1] - bounds[:, 0]
A = lo + rng.random((n_mc, d)) * span
B = lo + rng.random((n_mc, d)) * span
fA = np.asarray(model(A), float).ravel()
fB = np.asarray(model(B), float).ravel()
tot = float(np.var(np.concatenate([fA, fB])))
if tot <= _TINY:
return np.zeros(d), np.zeros(d)
s1 = np.zeros(d)
eff_var = np.zeros(d)
for i in range(d):
Bi = B.copy()
Bi[:, i] = A[:, i]
fBi = np.asarray(model(Bi), float).ravel()
s1[i] = float(np.mean((fB - fBi) ** 2)) / (2.0 * tot)
# 主效应：沿第 i 维把其它维固定在 A 上扫一遍，取响应的方差
grid = np.tile(A, (1, 1))
sweep = np.linspace(0.0, 1.0, 21)
curve = []
for t in sweep:
g = grid.copy()
g[:, i] = lo[i] + t * span[i]
curve.append(float(np.mean(model(g))))
eff_var[i] = float(np.var(curve))
return np.clip(s1, 0.0, 1.0), eff_var
def anova_variance_shares(model: Callable[[np.ndarray], np.ndarray],
bounds: Sequence[Tuple[float, float]],
n_mc: int = 4096,
seed: int = 0) -> np.ndarray:
"""把一阶指数归一化为"方差贡献占比（%）"，便于与论文 04/05 的 ANOVA 结论对照。"""
s1, _ = first_order_effects(model, bounds, n_mc=n_mc, seed=seed)
s = float(np.sum(s1))
if s <= _TINY:
return np.zeros_like(s1)
return 100.0 * s1 / s
# ---------------------------------------------------------------------- 自检
def _self_test() -> None:
rng = np.random.default_rng(20240905)
# 用一个 2 维解析函数做真值
def camel6(x: np.ndarray) -> np.ndarray:
x = np.atleast_2d(np.asarray(x, float))
x1, x2 = x[:, 0], x[:, 1]
return (4.0 - 2.1 * x1 ** 2 + x1 ** 4 / 3.0) * x1 ** 2 + x1 * x2 + (-4.0 + 4.0 * x2 ** 2) * x2 ** 2
bounds = [(-3.0, 3.0), (-2.0, 2.0)]
lo = np.array([b[0] for b in bounds])
span = np.array([b[1] - b[0] for b in bounds])
# 1) 采样拟合
n = 40
X = lo + rng.random((n, 2)) * span
y = camel6(X)
gp = Kriging(nugget=1e-8, n_restarts=2, seed=1).fit(X, y)
# 2) 插值性质：训练点处预测应几乎等于观测值（用响应量程做相对容差）
mu_tr, sd_tr = gp.predict(X)
rel_err = float(np.max(np.abs(mu_tr - y)) / (np.max(y) - np.min(y)))
assert rel_err < 1e-3, f"Kriging 未通过训练点（插值性质失效，相对误差 {rel_err:.2e}）"
assert np.all(sd_tr >= 0), "预测方差出现负值"
# 3) 泛化：独立测试集 R^2 应明显为正
Xt = lo + rng.random((500, 2)) * span
yt = camel6(Xt)
r2 = gp.r2(Xt, yt)
assert r2 > 0.8, f"测试集 R^2 过低：{r2:.3f}"
```

```
    # 4) 远离训练数据的地方，预测方差应大于近邻处
    sd_far = gp.predict(np.array([[2.9, 1.9]]))
    sd_near = gp.predict(X[:1])
    assert sd_far[0] > sd_near[0], "外推区不确定性未增大"
    # 5) 留一误差应小于响应量级
    loo = gp.leave_one_out_rmse()
    assert loo < 0.5 * (yt.max() - yt.min()), f"留一 RMSE 过大：{loo}"
    # 6) EI：在当前最优处为 0，且非负；GEI(g=1) 应与 EI 一致
    fbest = float(np.min(y))
    mu_t, sd_t = gp.predict(Xt)
    ei = expected_improvement(mu_t, sd_t, fbest)
    assert np.all(ei >= -1e-12), "EI 出现负值"
    ei_at_best = expected_improvement(np.array([fbest]), np.array([1e-6]), fbest)[0]
    assert abs(ei_at_best) < 1e-6, "EI 在当前最优处应为 0"
    gei1 = generalized_expected_improvement(mu_t, sd_t, fbest, g=1)
    assert np.allclose(ei, gei1, atol=1e-8), "GEI(g=1) 与 EI 不一致"
    gei2 = generalized_expected_improvement(mu_t, sd_t, fbest, g=2)
    assert np.all(gei2 >= -1e-12), "GEI(g=2) 出现负值"
    # 7) 一阶效应：camel6 对 x1 更敏感（x1 的项量级更大）
    s1, _ = first_order_effects(lambda Z: gp.predict(Z)[0], bounds, n_mc=512, seed=3)
    assert s1[0] > s1[1], f"一阶效应排序不符：{s1}"
    shares = anova_variance_shares(lambda Z: gp.predict(Z)[0], bounds, n_mc=512, seed=3)
    assert abs(float(np.sum(shares)) - 100.0) < 1e-6, "占比未归一化到 100%"
    # 8) 异常输入
    try:
        Kriging().fit(np.zeros((5, 2)), np.zeros(3))
    except ValueError:
        pass
    else:
        raise AssertionError("样本数不匹配未报错")
    print(f"[gp.py] 自检通过：插值性 R²={r2:.3f} / 留一RMSE={loo:.3f} / "
          f"EI-GEI 一致 / 一阶效应 S1={np.round(s1, 3)}")
if __name__ == "__main__":
    _self_test()
```

code/metrics.py

```
"""metrics.py —— 叶轮机械气动与传热评价指标

本全集所有"优化目标"最终都落在几个物理量上。要读懂各篇论文的表格，必须先把这些
指标的定义钉死。本模块给出**行业内通行的教科书定义**（不是某篇论文独创的公式），
并统一处理"面平均 / 质量平均""相对变化量""不确定度汇总"这三件最容易出错的事。

覆盖的指标与出处对照：
* **总压损失系数** ω —— 第 17 讲（论文 09）报告"出口马赫数 0.8 时面积平均总压
  损失系数下降 14.0%"；第 03 讲（论文 07）用等熵效率评价 126 维叶栅优化
* **面积平均气膜冷却有效度** η_aw、**面积平均努塞尔数** Nu —— 第 19 讲（论文 04）
  UQ 的两个热学输出："面积平均气膜冷却效率最大偏差 45%，面积平均 Nu 最大偏差 5.0%"
* **面积平均综合冷却效率** φ 及其**标准差** —— 第 20 讲（论文 05）三个评价指标中的
  前两个（第三个是总压降系数）
* **级等熵效率** —— 第 18 讲（论文 10）叶尖优化的目标函数

> ⚠ **关于"面平均"的诚实提醒**：论文里说"面积平均"，实务中有按**几何面积**加权和
> 按**质量流量**加权两种做法，二者在强三维流场里差异可观。本模块两者都提供
> （`area_average` / `mass_average`），请在使用时明确用的是哪一种——这是本全集
> 第 19 讲反复强调的"可复现性"底线。

**只依赖 numpy**。运行自检：
    python code/metrics.py
"""

from __future__ import annotations

from typing import Dict, Sequence

import numpy as np

__all__ = [
    "area_average", "mass_average", "area_average_with_std",
    "total_pressure_loss_coefficient", "static_pressure_coefficient",
    "adiabatic_film_effectiveness", "overall_cooling_effectiveness",
    "nusselt_number", "heat_transfer_coefficient",
    "isentropic_efficiency", "compressor_efficiency",
    "relative_change", "percentage_point_change",
    "describe", "deviation_band",
]

_TINY = 1e-12


# ---------------------------------------------------------------------- 平均方式

def area_average(values: np.ndarray, areas: np.ndarray) -> float:
    """面积（或任意权重）平均：Σ w_i · v_i / Σ w_i。"""
    v = np.asarray(values, float).ravel()
    w = np.asarray(areas, float).ravel()
    if v.size != w.size:
        raise ValueError("values 与 areas 的长度不一致")
    if np.any(w < 0):
        raise ValueError("权重（面积）不能为负")
    s = float(np.sum(w))
    if s <= _TINY:
        raise ValueError("权重（面积）之和为 0")
    return float(np.sum(w * v) / s)


def mass_average(values: np.ndarray, mass_flows: np.ndarray) -> float:
    """质量流量加权平均：Σ ṁ_i · v_i / Σ ṁ_i。

    与 `area_average` 的区别只在权重的物理含义；接口分开是为了避免混用。
    """
    v = np.asarray(values, float).ravel()
    m = np.asarray(mass_flows, float).ravel()
    if v.size != m.size:
```

```
    raise ValueError("values 与 mass_flows 的长度不一致")
    if np.any(m < 0):
        raise ValueError("质量流量不能为负")
    s = float(np.sum(m))
    if s <= _TINY:
        raise ValueError("质量流量之和为 0")
    return float(np.sum(m * v) / s)


def area_average_with_std(values: np.ndarray, areas: np.ndarray) -> tuple:
    """面积平均及其（面积加权的）标准差——对应论文 05 的"综合冷却效率 + 其标准差"
    这对指标。返回 (均值, 标准差)。"""
    v = np.asarray(values, float).ravel()
    w = np.asarray(areas, float).ravel()
    mean = area_average(v, w)
    var = area_average((v - mean) ** 2, w)
    return mean, float(np.sqrt(max(var, 0.0)))


# ---------------------------------------------------------------------- 气动指标
def total_pressure_loss_coefficient(pt_in: float, pt_out: float,
                                    ps_in: float) -> float:
    """总压损失系数 ω = (p_t,in − p_t,out) / (p_t,in − p_s,in)。

    分母用**进口动压头**，所以 ω 越接近 0 越好；若出口总压高于进口（有做功或数值
    异常），ω 为负，此时应回头检查而不是直接采信。
    """
    denom = float(pt_in) - float(ps_in)
    if abs(denom) <= _TINY:
        raise ValueError("进口动压头为 0，无法定义总压损失系数")
    return (float(pt_in) - float(pt_out)) / denom


def static_pressure_coefficient(p: float, p_ref: float, q_ref: float) -> float:
    """静压系数 Cp = (p − p_ref) / q_ref（q_ref 为参考动压）。

    第 17 讲（论文 09）用它刻画非轴对称端壁对端壁静压的改变。
    """
    if abs(float(q_ref)) <= _TINY:
        raise ValueError("参考动压不能为 0")
    return (float(p) - float(p_ref)) / float(q_ref)


def isentropic_efficiency(pt_in: float, pt_out: float,
                          Tt_in: float, Tt_out: float,
                          gamma: float = 1.4) -> float:
    """涡轮等熵效率 η = (1 − T_t,out/T_t,in) / (1 − (p_t,out/p_t,in)^((γ−1)/γ))。

    分子是**实际焓降**，分母是**等熵焓降**，故 η ∈ (0,1)。
    """
    Tt_in = float(Tt_in)
    if Tt_in <= _TINY:
        raise ValueError("进口总温必须为正")
    pr = float(pt_out) / float(pt_in)
    if pr <= 0 or pr > 1.0:
        raise ValueError("涡轮的出口总压应低于进口总压（压比 ⩽ 1）")
    actual = 1.0 - float(Tt_out) / Tt_in
    ideal = 1.0 - pr ** ((gamma - 1.0) / gamma)
    if abs(ideal) <= _TINY:
        raise ValueError("等熵焓降趋近 0，效率无定义")
    return actual / ideal


def compressor_efficiency(pt_in: float, pt_out: float,
                          Tt_in: float, Tt_out: float,
                          gamma: float = 1.4) -> float:
    """压气机等熵效率 η = (PR^((γ−1)/γ) − 1) / (T_t,out/T_t,in − 1)。"""
    pr = float(pt_out) / float(pt_in)
    if pr < 1.0:
        raise ValueError("压气机的压比应 ⩾ 1")
    ideal = pr ** ((gamma - 1.0) / gamma) - 1.0
    actual = float(Tt_out) / float(Tt_in) - 1.0
    if abs(actual) <= _TINY:
        raise ValueError("实际温升趋近 0，效率无定义")
    return ideal / actual


# ---------------------------------------------------------------------- 传热指标
def adiabatic_film_effectiveness(T_inf: float, T_aw: float, T_c: float) -> float:
    """绝热气膜冷却有效度 η_aw = (T_∞ − T_aw) / (T_∞ − T_c)。

    T_∞ 主流温度、T_aw 绝热壁温、T_c 冷气温度。
    η=0 表示气膜完全没起作用（壁温等于主流温度），η=1 表示壁温被压到冷气温度。
    """
    denom = float(T_inf) - float(T_c)
    if abs(denom) <= _TINY:
        raise ValueError("主流与冷气温差为 0，气膜有效度无定义")
    return (float(T_inf) - float(T_aw)) / denom


def overall_cooling_effectiveness(T_inf: float, T_w: float, T_c: float) -> float:
    """综合冷却效率 φ = (T_∞ − T_w) / (T_∞ − T_c)。

    与 η_aw 的区别：φ 用的是**真实壁温**（含内部冷却），η_aw 用的是**绝热壁温**
    （只含外部气膜）。第 20 讲（论文 05）的三个指标之一就是"面积平均综合冷却效率"。
    """
    denom = float(T_inf) - float(T_c)
    if abs(denom) <= _TINY:
        raise ValueError("主流与冷气温差为 0，综合冷却效率无定义")
    return (float(T_inf) - float(T_w)) / denom


def nusselt_number(h: float, L: float, k: float) -> float:
    """努塞尔数 Nu = h · L / k（h 对流换热系数、L 特征长度、k 流体导热系数）。"""
    if abs(float(k)) <= _TINY:
        raise ValueError("导热系数不能为 0")
    return float(h) * float(L) / float(k)


def heat_transfer_coefficient(nu: float, L: float, k: float) -> float:
    """由 Nu 反算对流换热系数 h = Nu · k / L（`nusselt_number` 的逆运算）。"""
    if abs(float(L)) <= _TINY:
        raise ValueError("特征长度不能为 0")
    return float(nu) * float(k) / float(L)
```

```
# ---------------------------------------------------------------------- 变化量
def relative_change(new: float, base: float) -> float:
"""相对变化量（%）：(new − base)/|base| × 100。
> 强烈建议统一用这个函数，而不是随手写 `a/b - 1`。第 03 讲（论文 07）里
> "GSDE 拿到 DE 约 99% 的效率增益"就是 1.104/1.115 这种**比值**口径，而
> "总压损失下降 14.0%"是**相对变化**口径，两者不能混为一谈。
"""
if abs(float(base)) <= _TINY:
raise ValueError("基准值为 0，相对变化量无定义")
return 100.0 * (float(new) - float(base)) / abs(float(base))
def percentage_point_change(new: float, base: float) -> float:
"""百分点变化：直接相减（用于本身就是百分比的量，如效率 91.2% → 91.6%）。"""
return float(new) - float(base)
def deviation_band(mu: float, sigma: float, k: float = 2.0) -> tuple:
"""不确定度区间 [μ − kσ, μ + kσ]（k=2 约对应 95% 置信度）。
第 19 讲（论文 04）的 UQ 输出需要这种"均值 ± 偏差"的表述。
"""
mu = float(mu)
s = abs(float(sigma)) * abs(float(k))
return (mu - s, mu + s)
def describe(values: np.ndarray, weights: np.ndarray = None) -> Dict[str, float]:
"""对一组样本给出统一的统计摘要（可选按面积加权）。
返回 dict：n / mean / std / min / max / rsd（相对标准差 %）/ p05 / p95。
"""
v = np.asarray(values, float).ravel()
if v.size == 0:
raise ValueError("输入为空")
if weights is None:
mean = float(np.mean(v))
std = float(np.std(v))
else:
mean, std = area_average_with_std(v, weights)
p05, p95 = np.percentile(v, [5, 95])
return {
"n": int(v.size),
"mean": mean,
"std": std,
"min": float(np.min(v)),
"max": float(np.max(v)),
"rsd": (100.0 * std / abs(mean)) if abs(mean) > _TINY else float("nan"),
"p05": float(p05),
"p95": float(p95),
}
# ---------------------------------------------------------------------- 自检
def _self_test() -> None:
rng = np.random.default_rng(5)
# 1) 面积平均：常数场 → 均值等于该常数；两块不同面积 → 手算核对
assert abs(area_average(np.full(10, 3.0), np.ones(10)) - 3.0) < 1e-12
assert abs(area_average(np.array([1.0, 2.0]), np.array([3.0, 1.0])) - 1.25) < 1e-12
assert abs(mass_average(np.array([1.0, 2.0]), np.array([3.0, 1.0])) - 1.25) < 1e-12
mu, sd = area_average_with_std(np.array([1.0, 3.0]), np.array([1.0, 1.0]))
assert abs(mu - 2.0) < 1e-12 and abs(sd - 1.0) < 1e-12, "面积加权均值/标准差错误"
assert area_average_with_std(np.full(5, 2.0), np.ones(5))[1] == 0.0, "常数场标准差应为 0"
# 2) 总压损失系数：无损失为 0，全损失为 1
assert abs(total_pressure_loss_coefficient(100.0, 100.0, 90.0)) < 1e-12
assert abs(total_pressure_loss_coefficient(100.0, 90.0, 90.0) - 1.0) < 1e-12
# 第 17 讲论文 09 的 -14.0%：损失系数下降 14% 是"相对变化"口径
w_ref, w_new = 0.0500, 0.0500 * 0.86
assert abs(relative_change(w_new, w_ref) + 14.0) < 1e-9, "相对变化口径错误"
# 3) 静压系数
assert abs(static_pressure_coefficient(101325.0, 100000.0, 5000.0) - 0.265) < 1e-3
# 4) 气膜有效度：上下界与中点
Tinf, Tc = 1500.0, 600.0
assert abs(adiabatic_film_effectiveness(Tinf, Tinf, Tc)) < 1e-12, "无气膜时应为 0"
assert abs(adiabatic_film_effectiveness(Tinf, Tc, Tc) - 1.0) < 1e-12, "完美气膜时应为 1"
assert abs(adiabatic_film_effectiveness(Tinf, 1050.0, Tc) - 0.5) < 1e-12
# 综合冷却效率用真实壁温：内部冷却把壁温压到 900K → φ=0.667
assert abs(overall_cooling_effectiveness(Tinf, 900.0, Tc) - 2.0 / 3.0) < 1e-12
# 同一壁温下，φ 一定 ⩾ η_aw（绝热壁温不低于真实壁温）
assert overall_cooling_effectiveness(Tinf, 900.0, Tc) >= \
adiabatic_film_effectiveness(Tinf, 1000.0, Tc)
# 5) Nu 与 h 互为逆运算
h, L, k = 850.0, 0.02, 0.06
nu = nusselt_number(h, L, k)
assert abs(nu - 283.3333333) < 1e-4, f"Nu 计算错误：{nu}"
assert abs(heat_transfer_coefficient(nu, L, k) - h) < 1e-9, "h 反算不一致"
# 6) 等熵效率：用一组自洽的数据，η 必须落在 (0,1)
# 自洽算例：p_t 1000→550 kPa、T_t,in = 1600 K、γ=1.33 时等熵温降约 124 K，
# 取实际温降 200 K（对应出口总温 1400 K）→ η ≈ 0.90
eta = isentropic_efficiency(1000.0, 550.0, 1600.0, 1400.0, gamma=1.33)
assert 0.0 < eta < 1.0, f"等熵效率超出 (0,1)：{eta}"
# 等熵过程（实际温降 = 理想温降）时效率应为 1
Tout_ideal = 1600.0 * (550.0 / 1000.0) ** ((1.33 - 1.0) / 1.33)
assert abs(isentropic_efficiency(1000.0, 550.0, 1600.0, Tout_ideal, 1.33) - 1.0) < 1e-9
# 压气机
eta_c = compressor_efficiency(100.0, 300.0, 288.0, 480.0, 1.4)
assert 0.0 < eta_c < 1.0, f"压气机效率超出 (0,1)：{eta_c}"
# 7) 百分点 vs 相对变化（第 18 讲论文 10 的 +0.42% 是"效率增量"，属百分点口径）
assert abs(percentage_point_change(91.62, 91.20) - 0.42) < 1e-9
assert abs(relative_change(91.62, 91.20) - 0.4605) < 1e-3
# 8) 不确定度区间与 describe
```

```
lo  b, hi  b = deviation  band(0.35, 0.05, k=2.0)
assert abs(lo  b - 0.25) < 1e-12 and abs(hi  b - 0.45) < 1e-12
x = rng.normal(10.0, 2.0, size=5000)
s = describe(x)
assert s["n"] == 5000 and abs(s["mean"] - 10.0) < 0.2 and abs(s["std"] - 2.0) < 0.2
assert s["min"] <= s["p05"] <= s["p95"] <= s["max"]
sw = describe(x, weights=np.ones  like(x))
assert abs(sw["mean"] - s["mean"]) < 1e-9, "等权重的加权均值应等于算术均值"
# 9) 异常输入
for bad in [lambda: area  average(np.ones(3), np.ones(4)),
lambda: area  average(np.ones(3), np.zeros(3)),
lambda: total  pressure  loss  coefficient(100.0, 90.0, 100.0),
lambda: adiabatic  film  effectiveness(600.0, 800.0, 600.0),
lambda: isentropic  efficiency(550.0, 1000.0, 1600.0, 1250.0),
lambda: compressor  efficiency(300.0, 100.0, 288.0, 480.0),
lambda: relative  change(1.0, 0.0),
lambda: describe(np.zeros(0))]:
try:
bad()
except ValueError:
pass
else:
raise AssertionError("非法输入未报错")
print(f"[metrics.py] 自检通过：平均/损失系数/气膜有效度/Nu/等熵效率"
f"(η_t={eta:.4f}, η_c={eta_c:.4f})/变化量口径/统计摘要/异常输入")
if  name  == "  main  ":
_self_test()
```

code/moo.py

```
"""moo.py —— 多目标优化：非支配排序、拥挤距离、NSGA-II 与性能评价指标

本全集中有两处明确使用 **NSGA-II** 做多目标优化：
* **论文 16（第 10 讲，TNO）**：NSGA-II，种群 30、代数 100「原文 P16 Table 10」
* **论文 15（第 14 讲，流固耦合数字孪生）**：NSGA-II，种群 100、每代 10 个子代，
以 POD 模态系数为设计变量，优化目标为效率与最大应力「原文 P15 §III」
此外，第 16 讲与第 20 讲在"审稿人批判"里都指出：**只做若干次单目标优化，无法刻画
目标之间的权衡曲面（Pareto 前沿）**，因此"最佳折中方案"其实没被真正找到
「原文 P16 §局限；原文 P05 讨论」。多目标模块因此是本代码库必要的一环。

本模块提供：
* `dominates` / `non  dominated  sort` / `crowding  distance` —— NSGA-II 的两个核心算子
* `nsga2` —— 完整的多目标进化算法（SBX 交叉 + 多项式变异 + (μ+λ) 精英保留）
* `hypervolume  2d` / `hypervolume  mc` —— 超体积指标（2 目标精确、多目标蒙特卡洛）
* `igd` / `spread` —— 与参考前沿的距离与分布均匀性
* `pareto_front` —— 从一堆解里筛出非支配解

**只依赖 numpy**。运行自检：
python code/moo.py
"""

from __future__ import annotations

from typing import Callable, List, Optional, Sequence, Tuple

import numpy as np

  all  = [
"dominates", "non  dominated  sort", "crowding  distance", "pareto  front",
"nsga2", "hypervolume  2d", "hypervolume  mc", "igd", "spread",
]

_TINY = 1e-12

# ---------------------------------------------------------------------------- 支配关系

def dominates(a: Sequence[float], b: Sequence[float]) -> bool:
"""（最小化意义下）a 支配 b：a 在所有目标上不劣于 b，且至少一维严格更优。"""
a = np.asarray(a, float).ravel()
b = np.asarray(b, float).ravel()
if a.size != b.size:
raise ValueError("目标维数不一致")
return bool(np.all(a <= b) and np.any(a < b))

def non  dominated  sort(F: np.ndarray) -> Tuple[np.ndarray, List[List[int]]]:
"""快速非支配排序（NSGA-II 原文算法）。

返回 (rank, fronts)：rank[i] 是个体 i 所属前沿编号（0 为最优前沿），
fronts[k] 是第 k 层前沿的个体下标列表。
"""
F = np.asarray(F, float)
n = F.shape[0]
if n == 0:
return np.zeros(0, dtype=int), []
# 暴力 O(n²m)：n 为种群规模（几十~几百），完全够用，代码也最直白
dom  sets: List[List[int]] = [[] for  in range(n)] # i 支配了谁
be  dominated = np.zeros(n, dtype=int) # 有多少个体支配 i
for i in range(n):
for j in range(i + 1, n):
if dominates(F[i], F[j]):
dom  sets[i].append(j)
be  dominated[j] += 1
elif dominates(F[j], F[i]):
dom  sets[j].append(i)
be_dominated[i] += 1

rank = np.full(n, -1, dtype=int)
fronts: List[List[int]] = []
cur = [i for i in range(n) if be  dominated[i] == 0]
k = 0
while cur:
fronts.append(sorted(cur))
```

```
for i in cur:
rank[i] = k
nxt: List[int] = []
for i in cur:
for j in dom_sets[i]:
be_dominated[j] -= 1
if be_dominated[j] == 0:
nxt.append(j)
cur = nxt
k += 1
return rank, fronts
def crowding_distance(F: np.ndarray, front: Sequence[int]) -> np.ndarray:
"""同一前沿内的拥挤距离（边界个体记为 +inf，保证多样性）。
距离越大代表该个体周围越"空旷"，越应被保留。
"""
F = np.asarray(F, float)
idx = list(front)
m = len(idx)
dist = np.zeros(m)
if m <= 2:
dist[:] = np.inf
return dist
sub = F[idx]
rng = sub.max(axis=0) - sub.min(axis=0)
for j in range(F.shape[1]):
order = np.argsort(sub[:, j])
dist[order[0]] = np.inf
dist[order[-1]] = np.inf
span = rng[j]
if span <= _TINY:
continue
for t in range(1, m - 1):
dist[order[t]] += (sub[order[t + 1], j] - sub[order[t - 1], j]) / span
return dist
def pareto_front(F: np.ndarray) -> np.ndarray:
"""返回非支配解的下标（第 0 层前沿）。"""
_, fronts = non_dominated_sort(F)
return np.array(fronts[0], dtype=int)
# ---------------------------------------------------------------------- 遗传算子
def _sbx(p1: np.ndarray, p2: np.ndarray, eta: float, rng: np.random.Generator,
lo: np.ndarray, hi: np.ndarray) -> Tuple[np.ndarray, np.ndarray]:
"""模拟二进制交叉 SBX。"""
u = rng.random(p1.size)
beta = np.where(u <= 0.5,
(2.0 * u) ** (1.0 / (eta + 1.0)),
(1.0 / (2.0 * (1.0 - u))) ** (1.0 / (eta + 1.0)))
c1 = 0.5 * ((1 + beta) * p1 + (1 - beta) * p2)
c2 = 0.5 * ((1 - beta) * p1 + (1 + beta) * p2)
return np.clip(c1, lo, hi), np.clip(c2, lo, hi)
def _poly_mutate(x: np.ndarray, eta: float, pm: float, rng: np.random.Generator,
lo: np.ndarray, hi: np.ndarray) -> np.ndarray:
"""多项式变异。"""
out = x.copy()
mask = rng.random(x.size) < pm
if not np.any(mask):
return out
u = rng.random(np.sum(mask))
delta = np.where(u < 0.5,
(2.0 * u) ** (1.0 / (eta + 1.0)) - 1.0,
1.0 - (2.0 * (1.0 - u)) ** (1.0 / (eta + 1.0)))
out[mask] = out[mask] + delta * (hi[mask] - lo[mask])
return np.clip(out, lo, hi)
# ---------------------------------------------------------------------- NSGA-II
def nsga2(func: Callable[[np.ndarray], np.ndarray],
bounds: Sequence[Tuple[float, float]],
pop_size: int = 100,
n_gen: int = 100,
n_offspring: Optional[int] = None,
eta_c: float = 20.0,
eta_m: float = 20.0,
p_cross: float = 0.9,
seed: int = 0,
return_history: bool = False):
"""NSGA-II 主循环 （(μ+λ) 精英保留）。
参数
----
func : 目标函数，输入 (d,) 或 (n,d)，输出 (m,) 或 (n,m)；统一按**最小化**
bounds : 变量上下界
pop_size : 种群规模 μ（论文 16 用 30，论文 15 用 100）
n_gen : 代数（论文 16 用 100 代）
n_offspring : 每代子代数 λ，默认 = pop_size（论文 15 用每代 10 个子代）
return_history : 是否返回每代的最优目标值轨迹
返回 (X, F) 或 (X, F, history)。
"""
rng = np.random.default_rng(seed)
bounds = np.asarray(bounds, float)
lo, hi = bounds[:, 0], bounds[:, 1]
d = bounds.shape[0]
lam = int(n_offspring) if n_offspring else int(pop_size)
pm = 1.0 / d
def evaluate(P: np.ndarray) -> np.ndarray:
F = np.asarray(func(P), float)
```

```
if F.ndim == 1:
F = F.reshape(1, -1)
return F
# 初始化：均匀随机种群
P = lo + rng.random((pop_size, d)) * (hi - lo)
F = evaluate(P)
m = F.shape[1]
history: List[float] = []
for _ in range(n_gen):
# --- 繁殖
children = []
while len(children) < lam:
i, j = rng.integers(0, pop_size, size=2)
if rng.random() < p_cross:
c1, c2 = _sbx(P[i], P[j], eta_c, rng, lo, hi)
else:
c1, c2 = P[i].copy(), P[j].copy()
children.append(_poly_mutate(c1, eta_m, pm, rng, lo, hi))
if len(children) < lam:
children.append(_poly_mutate(c2, eta_m, pm, rng, lo, hi))
C = np.array(children[:lam])
FC = evaluate(C)
# --- (μ+λ) 合并后按 非支配层级 → 拥挤距离 排序截断
Pall = np.vstack([P, C])
Fall = np.vstack([F, FC])
rank, fronts = non_dominated_sort(Fall)
keep: List[int] = []
for fr in fronts:
if len(keep) + len(fr) <= pop_size:
keep.extend(fr)
else:
cd = crowding_distance(Fall, fr)
order = np.argsort(-cd) # 拥挤距离大的优先
need = pop_size - len(keep)
keep.extend(np.array(fr)[order][:need].tolist())
break
P, F = Pall[keep], Fall[keep]
if return_history:
history.append(float(np.min(F[:, 0])))
return (P, F, np.array(history)) if return_history else (P, F)
# ---------------------------------------------------------------------- 评价指标
def hypervolume_2d(F: np.ndarray, ref: Sequence[float]) -> float:
"""2 目标超体积（精确，梯形累加法）。ref 为被所有解支配的参考点。"""
F = np.asarray(F, float)
ref = np.asarray(ref, float).ravel()
if F.size == 0:
return 0.0
front = F[pareto_front(F)]
front = front[np.argsort(front[:, 0])]
hv = 0.0
for k in range(len(front)):
width = (ref[0] - front[k, 0]) if k == len(front) - 1 \
else (front[k + 1, 0] - front[k, 0])
hv += width * (ref[1] - front[k, 1])
return float(max(hv, 0.0))
def hypervolume_mc(F: np.ndarray, ref: Sequence[float], n_mc: int = 100000,
seed: int = 0) -> float:
"""任意目标数的超体积（蒙特卡洛估计）：在「最优点, ref」盒子里随机撒点，
统计被至少一个解支配的比例，乘以盒子体积。"""
F = np.asarray(F, float)
ref = np.asarray(ref, float).ravel()
if F.size == 0:
return 0.0
lo = np.minimum(F.min(axis=0), ref)
box = float(np.prod(ref - lo))
if box <= _TINY:
return 0.0
rng = np.random.default_rng(seed)
pts = lo + rng.random((n_mc, F.shape[1])) * (ref - lo)
hit = np.zeros(n_mc, dtype=bool)
for i in range(F.shape[0]):
hit |= np.all(F[i] <= pts, axis=1)
return float(box * np.mean(hit))
def igd(F: np.ndarray, ref_front: np.ndarray) -> float:
"""反世代距离 IGD：参考前沿上每个点到最近解的平均距离（越小越好）。"""
F = np.asarray(F, float)
R = np.asarray(ref_front, float)
if F.size == 0 or R.size == 0:
return float("inf")
d2 = np.sum((R[:, None, :] - F[None, :, :]) ** 2, axis=2)
return float(np.mean(np.sqrt(np.min(d2, axis=1))))
def spread(F: np.ndarray) -> float:
"""分布均匀性（越小越均匀）：前沿上相邻点距离的标准差除以均值。
仅对 2 目标有意义；目标数 >2 时退化为对第一维排序后的相邻距离统计。
"""
front = np.asarray(F, float)
if front.shape[0] < 3:
return 0.0
front = front[np.argsort(front[:, 0])]
gaps = np.linalg.norm(np.diff(front, axis=0), axis=1)
mean = float(np.mean(gaps))
if mean <= _TINY:
```

```
return 0.0
return float(np.std(gaps) / mean)
# ---------------------------------------------------------------------- 自检
def _self_test() -> None:
rng = np.random.default_rng(20240905)
# 1) 支配关系
assert dominates([1, 1], [2, 2]), "支配判定错误"
assert not dominates([1, 2], [2, 1]), "不可比却判为支配"
assert not dominates([1, 1], [1, 1]), "相同点不应互相支配"
assert not dominates([2, 2], [1, 1]), "支配方向反了"
# 2) 非支配排序：构造三层前沿
F = np.array([[1.0, 5.0], [2.0, 4.0], [3.0, 3.0], [4.0, 2.0], [5.0, 1.0],
[2.0, 6.0], [3.0, 5.0], [6.0, 6.0]])
rank, fronts = non_dominated_sort(F)
assert len(fronts) == 3, f"前沿层数应为 3，实为 {len(fronts)}"
assert fronts[0] == [0, 1, 2, 3, 4], f"第一层前沿不对：{fronts[0]}"
assert fronts[1] == [5, 6], f"第二层前沿不对：{fronts[1]}"
assert fronts[2] == [7], f"第三层前沿不对：{fronts[2]}"
assert np.array_equal(rank, [0, 0, 0, 0, 0, 1, 1, 2]), "rank 数组不对"
assert set(pareto_front(F).tolist()) == {0, 1, 2, 3, 4}, "pareto_front 不对"
# 3) 拥挤距离：边界必须最大
cd = crowding_distance(F, fronts[0])
assert np.isinf(cd[0]) and np.isinf(cd[-1]), "边界个体拥挤距离应为 inf"
assert cd[2] > 0, "中间个体拥挤距离应为正"
# 4) 超体积：加入更优解后 HV 必须变大；MC 估计应接近 2 目标精确值
ref = [6.5, 6.5]
hv1 = hypervolume_2d(np.array([[3.0, 3.0]]), ref)
hv2 = hypervolume_2d(np.array([[3.0, 3.0], [2.0, 2.0]]), ref)
assert hv2 > hv1 > 0, "超体积单调性错误"
hvmc = hypervolume_mc(np.array([[3.0, 3.0], [2.0, 2.0]]), ref, n_mc=200000, seed=1)
assert abs(hvmc - hv2) / hv2 < 0.02, f"MC 超体积与精确值偏差过大：{hvmc} vs {hv2}"
# 5) IGD：解集越贴近参考前沿，IGD 越小；重复解不降低 IGD
ref_front = np.column_stack([np.linspace(0, 1, 21), 1 - np.linspace(0, 1, 21) ** 2])
good = ref_front.copy()
bad = ref_front + 0.5
assert igd(good, ref_front) < igd(bad, ref_front), "IGD 未反映贴近程度"
assert abs(igd(good, ref_front)) < 1e-9, "完全重合时 IGD 应为 0"
# 6) NSGA-II 真的能逼近已知 Pareto 前沿
# 经典双目标问题：f1 = x1，f2 = g · (1 - sqrt(x1/g))，g = 1 + 9 · mean(x2:)
def zdt1_like(P: np.ndarray) -> np.ndarray:
P = np.atleast_2d(np.asarray(P, float))
f1 = P[:, 0]
g = 1.0 + 9.0 * np.mean(P[:, 1:], axis=1)
f2 = g * (1.0 - np.sqrt(np.maximum(f1 / g, 0.0)))
return np.column_stack([f1, f2])
bounds = [(0.0, 1.0)] * 6
X, Fp, hist = nsga2(zdt1_like, bounds, pop_size=60, n_gen=120,
seed=7, return_history=True)
pf = Fp[pareto_front(Fp)]
assert pf.shape[0] >= 40, f"非支配解数量偏少：{pf.shape[0]}"
assert np.min(pf[:, 0]) < 0.05, "未能找到 f1 极小端"
assert np.min(pf[:, 1]) < 0.15, "未能找到 f2 极小端"
# 前沿上任意两点不应互相支配
for i in range(0, pf.shape[0], 7):
for j in range(0, pf.shape[0], 7):
if i != j:
assert not dominates(pf[i], pf[j]), "输出解集中存在互相支配的点"
ref_true = np.column_stack([np.linspace(0, 1, 51), 1 - np.sqrt(np.linspace(0, 1, 51))])
igd_now = igd(pf, ref_true)
assert igd_now < 0.05, f"IGD 偏大，前沿质量不足：{igd_now:.4f}"
assert spread(pf) < 2.0, f"前沿分布过于不均：{spread(pf):.3f}"
# 7) 进化过程应当是"有效"的：跑到 120 代的前沿应优于只跑 20 代
_, F_short, _ = nsga2(zdt1_like, bounds, pop_size=60, n_gen=20, seed=7,
return_history=True)
igd_short = igd(F_short[pareto_front(F_short)], ref_true)
assert igd_now < igd_short, "多跑代数后前沿质量没有提升"
# 8) 异常输入
try:
dominates([1, 2], [1, 2, 3])
except ValueError:
pass
else:
raise AssertionError("目标维数不一致未报错")
assert igd(np.zeros((0, 2)), ref_true) == float("inf"), "空解集 IGD 应为 inf"
assert hypervolume_2d(np.zeros((0, 2)), [1, 1]) == 0.0, "空解集 HV 应为 0"
print(f"[moo.py] 自检通过：排序3层 / 拥挤距离 / HV单调(MC误差{abs(hvmc - hv2) / hv2:.2%}) / "
f"NSGA-II 前沿 {pf.shape[0]} 点，IGD={igd_now:.4f}")
if __name__ == "__main__":
_self_test()
```

code/multifidelity.py

```
"""multifidelity.py —— 多保真度建模与"把钱花在刀刃上"的样本分流
多保真是本全集贯穿始终的主线之一。本模块实现其中**可复现、且原文给出明确公式**的
几件事：
| 函数 / 类 | 对应论文 | 说明 |
| :--- | :--- | :--- |
| `CoKriging` | 论文 02（原文结构）；论文 17 以之为对比基线；论文 13 的 MFS 属同类但公式未获取 | 自回归（AR1）Co-Kriging：y_HF = ρ · y_LF + Z_DF |
| `filter_gei_threshold` | **论文 02（Filter-GEI）式 10–12** | 自适应权重 ω 与分流阈值 T 的**原文公式** |
| `hierarchical_decluster` | **论文 02** 第四步 | 层次聚类去掉重叠候选点 |
```

`fidelity_split`	**论文 02** 第 6 步	预测优于 T 的送 HF，其余送 LF
`dbscan`	**论文 13（EMFS/MSFO）**	密度聚类找出"多保真代理建不准"的失真区
`EMFSEnsemble`	**论文 13（EMFS）**	MFS 与局部单保真代理的自适应加权融合

两处必须说清楚的诚实声明：
1. 论文 13 的 **DBSCAN 参数（ε、MinPts）与自适应权重公式原文未给出**（第 05 讲已标注"原文未获取，本讲不作推测"）。因此 `dbscan` 实现的是标准 DBSCAN 算法（公开算法），而 `EMFSEnsemble` 的权重公式是**本代码库自己的工程选择**，不是论文数值。
2. `CoKriging` 的预测方差采用**常用简化**：$s^2_{HF} = \rho^2 \cdot s^2_{LF} + s^2_{DF}$（把 LF 与差异函数两个 GP 视为相互独立）。严格做法需展开二者协方差，工程量与收益不成比例，在"用方差做探索"这一用途上该简化被广泛采用。

代理模型通过 `gp_factory` 注入（鸭子接口：`.fit(X,y)` → self，`.predict(Xs)` → (mean, sd)）。默认使用本模块内置的精简 Kriging（`TinyKriging`）；若要用 `code/gp.py` 的完整实现，传入 `gp_factory=lambda: Kriging()` 即可。**注意：code/ 下的模块互不 import，是刻意的零耦合设计**（每个模块都能单独拷贝出去用）。

只依赖 numpy。运行自检：
python code/multifidelity.py
"""

```
from __future__ import annotations
import math
from typing import Callable, List, Optional, Sequence, Tuple
import numpy as np

__all__ = [
"CoKriging", "filter_gei_threshold", "hierarchical_decluster",
"fidelity_split", "dbscan", "EMFSEnsemble", "hedge_choose",
]

_TINY = 1e-12
_ERF = np.vectorize(math.erf, otypes=[float])

# ---------------------------------------------------------------------------- 内置精简 Kriging
class TinyKriging:
"""模块内置的精简普通 Kriging（ARD 高斯相关 + 坐标下降搜超参）。
存在的唯一原因：让 `CoKriging` 开箱即用。需要更严谨的实现请用 `code/gp.py`。
"""
def __init__(self, nugget: float = 1e-8, seed: int = 0):
self.nugget = nugget

@staticmethod
def _corr(A: np.ndarray, B: np.ndarray, theta: np.ndarray) -> np.ndarray:
diff = A[:, None, :] - B[None, :, :]
return np.exp(-np.einsum("nmk,k->nm", diff ** 2, theta))

def _loglik(self, theta: np.ndarray) -> Tuple[float, Optional[dict]]:
R = self._corr(self.X, self.X, theta)
R[np.diag_indices_from(R)] += self.nugget
try:
L = np.linalg.cholesky(R)
except np.linalg.LinAlgError:
return -np.inf, None
one = np.ones(self.X.shape[0])
Ri1 = np.linalg.solve(L.T, np.linalg.solve(L, one))
Riy = np.linalg.solve(L.T, np.linalg.solve(L, self.y))
mu = float(one @ Riy) / float(one @ Ri1)
r = self.y - one * mu
s2 = float(r @ np.linalg.solve(L.T, np.linalg.solve(L, r))) / self.X.shape[0]
if s2 <= 0:
return -np.inf, None
ll = (-0.5 * self.X.shape[0] * math.log(s2)
- float(np.sum(np.log(np.diag(L)))))
return ll, {"L": L, "mu": mu, "s2": s2, "Ri1": Ri1, "Riy": Riy}

def fit(self, X: np.ndarray, y: np.ndarray) -> "TinyKriging":
X = np.atleast_2d(np.asarray(X, float))
y = np.asarray(y, float).ravel()
self.xmin = X.min(axis=0)
self.xspan = X.max(axis=0) - self.xmin
self.xspan[self.xspan <= 0] = 1.0
self.X = (X - self.xmin) / self.xspan
self.y_mean, self.y_std = float(np.mean(y)), float(np.std(y))
self.y_std = self.y_std if self.y_std > _TINY else 1.0
self.y = (y - self.y_mean) / self.y_std

d = self.X.shape[1]
cur = np.zeros(d)
best_ll, best = -np.inf, cur.copy()
for start in (np.log10(0.1), 0.0, 1.0):
cur = np.full(d, start)
for _ in range(2):
for k in range(d):
grid = np.linspace(-2.5, 2.5, 21)
vals = []
for v in grid:
t = cur.copy()
t[k] = v
ll, _ = self._loglik(10.0 ** t)
vals.append(ll)
cur[k] = grid[int(np.argmax(vals))]
ll, _ = self._loglik(10.0 ** cur)
if ll > best_ll:
best_ll, best = ll, cur.copy()
self.theta = 10.0 ** best
ll, aux = self._loglik(self.theta)
if aux is None:
self.theta = np.full(d, 0.1)
ll, aux = self._loglik(self.theta)
self.L, self.mu, self.sigma2 = aux["L"], aux["mu"], aux["s2"]
self.Ri1, self.Riy = aux["Ri1"], aux["Riy"]
```

```
self.loglik = ll
return self
def predict(self, Xs: np.ndarray) -> Tuple[np.ndarray, np.ndarray]:
Z = (np.atleast_2d(np.asarray(Xs, float)) - self.xmin) / self.xspan
mus, s2s = [], []
for z in Z:
r = self._corr(z.reshape(1, -1), self.X, self.theta).ravel()
Rir = np.linalg.solve(self.L.T, np.linalg.solve(self.L, r))
mus.append(self.mu + float(r @ (self.Riy - self.mu * self.Ri1)))
s2s.append(max(self.sigma2 * (1.0 + self.nugget - float(r @ Rir)), 0.0))
return (np.array(mus) * self.y_std + self.y_mean,
np.sqrt(np.array(s2s)) * self.y_std)
# ---------------------------------------------------------------------- Co-Kriging
class CoKriging:
"""自回归（AR1）Co-Kriging：y_HF(x) = ρ · y_LF(x) + Z_DF(x)。
本全集中与之直接对应的论文：
* 论文 02（Filter-GEI）用它做代理，并由 σ²_DF / σ²_LF 决定滤波阈值［原文 P02 §2–3］；
* 论文 17（GTO）把"co-kriging 型 MFS"作为与 EMFS 对照的多保真基线［原文 P17 §4］；
* 论文 13（EMFS/MSFO）以某种 MFS 为底、在失真区改建 SFS，但其 MFS 具体公式未获取
［出版商页面 摘要，论文 13］；论文 08（KT-ASO）用"含相关系数 ρ 的多保真代理"
把源任务样本当作低保真信息［原文 P08 术语表/§3.3］，与 AR1 形式一致但细节以原文为准。
用法：
ck = CoKriging().fit(X_lf, y_lf, X_hf, y_hf)
mu, sd = ck.predict(Xs)
print(ck.rho, ck.sigma2_lf, ck.sigma2_df)
"""
def __init__(self, gp_factory: Optional[Callable[[], object]] = None,
rho_grid: Sequence[float] = tuple(np.linspace(-2.0, 3.0, 51))):
self.gp_factory = gp_factory if gp_factory is not None else (lambda: TinyKriging())
self.rho_grid = np.asarray(rho_grid, float)
def fit(self, X_lf: np.ndarray, y_lf: np.ndarray,
X_hf: np.ndarray, y_hf: np.ndarray) -> "CoKriging":
X_lf = np.atleast_2d(np.asarray(X_lf, float))
X_hf = np.atleast_2d(np.asarray(X_hf, float))
y_lf = np.asarray(y_lf, float).ravel()
y_hf = np.asarray(y_hf, float).ravel()
if X_lf.shape[1] != X_hf.shape[1]:
raise ValueError("低保真与高保真样本的维度不一致")
if X_lf.shape[0] < 3 or X_hf.shape[0] < 2:
raise ValueError("样本太少：LF 至少 3 个、HF 至少 2 个")
self.gp_lf = self.gp_factory().fit(X_lf, y_lf)
self.sigma2_lf = float(np.var(y_lf)) if np.var(y_lf) > _TINY else _TINY
# 在 HF 点上取 LF 的预测，构造差异样本 d = y_HF − ρ · ŷ_LF
mu_lf_at_hf, _ = self.gp_lf.predict(X_hf)
def _fit_df(rho: float):
d = y_hf - float(rho) * mu_lf_at_hf
try:
gp_df = self.gp_factory().fit(X_hf, d)
except (ValueError, np.linalg.LinAlgError):
return None, -np.inf
return gp_df, getattr(gp_df, "loglik", -np.inf)
best_ll, best_rho, best_model = -np.inf, 1.0, None
for rho in self.rho_grid:
m, ll = _fit_df(rho)
if m is not None and ll > best_ll:
best_ll, best_rho, best_model = ll, float(rho), m
if best_model is None:
raise RuntimeError("Co-Kriging 拟合失败：所有 ρ 都无法建立差异模型")
# 在最优网格点的邻域内做黄金分割细化（网格步长可能较粗）
i = int(np.argmin(np.abs(self.rho_grid - best_rho)))
a = float(self.rho_grid[max(0, i - 1)])
b = float(self.rho_grid[min(len(self.rho_grid) - 1, i + 1)])
gr = (math.sqrt(5.0) - 1.0) / 2.0
c, e = b - gr * (b - a), a + gr * (b - a)
mc, llc = _fit_df(c)
me, lle = _fit_df(e)
for _ in range(12):
if llc > lle:
b, e, me, lle = e, c, mc, llc
c = b - gr * (b - a)
mc, llc = _fit_df(c)
else:
a, c, mc, llc = c, e, me, lle
e = a + gr * (b - a)
me, lle = _fit_df(e)
for cand, m, ll in ((c, mc, llc), (e, me, lle)):
if m is not None and ll > best_ll:
best_ll, best_rho, best_model = ll, float(cand), m
self.rho = best_rho
self.gp_df = best_model
self.sigma2_df = float(np.var(y_hf - best_rho * mu_lf_at_hf))
self.loglik = best_ll
return self
def predict(self, Xs: np.ndarray) -> Tuple[np.ndarray, np.ndarray]:
mu_lf, sd_lf = self.gp_lf.predict(Xs)
mu_df, sd_df = self.gp_df.predict(Xs)
mu = self.rho * mu_lf + mu_df
# 简化方差：把 LF 与差异函数两个 GP 视为独立（见模块文档声明）
sd = np.sqrt(np.maximum((self.rho ** 2) * sd_lf ** 2 + sd_df ** 2, 0.0))
return mu, sd
# ---------------------------------------------------------------------- Filter-GEI（论文 02）
```

```
def filter_gei_threshold(sigma2_df: float, sigma2_lf: float,
v_hf_min: float, v_hf_mean: float) -> Tuple[float, float]:
"""论文 02（Filter-GEI）的自适应权重与分流阈值［原文式 10–12］：
ω = 1 / (1 + (σ̂²_DF / σ̂²_LF)^0.5)
T = ω · y_HF-min + (1 − ω) · y_HF-mean
含义：LF 与 HF 越相关 → σ²_DF 越小 → ω→1 → T 贴近当前最优 → **只给非常好的点投 HF**；
相关性越差 → ω→0 → T 贴近历史均值 → 扩大 HF 投放区域、加强全局探索。
"""
if sigma2_lf <= _TINY:
raise ValueError("σ̂²_LF 必须为正")
if sigma2_df < 0:
raise ValueError("σ̂²_DF 不能为负")
omega = 1.0 / (1.0 + math.sqrt(sigma2_df / sigma2_lf))
T = omega * float(v_hf_min) + (1.0 - omega) * float(v_hf_mean)
return float(omega), float(T)
def hierarchical_decluster(X: np.ndarray, d_T: float) -> List[List[int]]:
"""层次聚类去重叠［论文 02 第四步］。
逐个合并距离最近的两类，直到**最小的类间距离大于阈值 d_T** 为止。返回每类的下标列表。
"""
X = np.atleast_2d(np.asarray(X, float))
n = X.shape[0]
if n == 0:
return []
if d_T < 0:
raise ValueError("距离阈值 d_T 不能为负")
def _dist(a: List[int], b: List[int]) -> float:
"""类间距离 = 两类间**最近**两点（单链接）。"""
A, B = X[a], X[b]
diff = A[:, None, :] - B[None, :, :]
return float(np.sqrt(np.min(np.einsum("nmk,nmk->nm", diff, diff))))
clusters: List[List[int]] = [[i] for i in range(n)]
while len(clusters) > 1:
best, bi, bj = np.inf, 0, 1
for i in range(len(clusters)):
for j in range(i + 1, len(clusters)):
dd = _dist(clusters[i], clusters[j])
if dd < best:
best, bi, bj = dd, i, j
if best > d_T:
break
merged = clusters[bi] + clusters[bj]
clusters = [c for k, c in enumerate(clusters) if k not in (bi, bj)] + [merged]
return [sorted(c) for c in clusters]
def fidelity_split(pred: np.ndarray, T: float) -> Tuple[np.ndarray, np.ndarray]:
"""按阈值 T 分流「论文 02 第 6 步」：预测值**优于** T（最小化意义下即 < T）的送 HF，
其余送 LF。返回 (hf_indices, lf_indices)。"""
pred = np.asarray(pred, float).ravel()
hf = np.where(pred < T)[0]
lf = np.where(pred >= T)[0]
return hf, lf
def budget_report(n_hf: int, n_lf: int, cost_ratio: float = 0.1) -> dict:
"""把"HF/LF 各用了多少次"折算成**等效 HF 次数**（论文 02 的工程算例中
LF 与 HF 的成本比为 0.1，一次 HF 的钱能做十次 LF）。"""
if cost_ratio <= 0:
raise ValueError("成本比必须为正")
return {"n_hf": int(n_hf), "n_lf": int(n_lf), "cost_ratio": float(cost_ratio),
"equivalent_hf": int(n_hf) + float(cost_ratio) * int(n_lf)}
# ---------------------------------------------------------------------------- DBSCAN（论文 13）
def dbscan(X: np.ndarray, eps: float, min_samples: int = 5) -> np.ndarray:
"""标准 DBSCAN 密度聚类（论文 13 用它识别"多保真代理建不准"的失真区）。
返回标签数组：-1 表示噪声点，其余为簇编号。
参数 ε 与 MinPts 的具体取值**原文未给出**，需按样本尺度自行标定。
"""
X = np.atleast_2d(np.asarray(X, float))
n = X.shape[0]
if eps <= 0:
raise ValueError("eps 必须为正")
if min_samples < 1:
raise ValueError("min_samples 至少为 1")
if n == 0:
return np.zeros(0, dtype=int)
diff = X[:, None, :] - X[None, :, :]
D = np.sqrt(np.einsum("nmk,nmk->nm", diff, diff))
labels = np.full(n, -2, dtype=int) # -2 未访问，-1 噪声
c = 0
for p in range(n):
if labels[p] != -2:
continue
seeds = list(np.where(D[p] <= eps)[0])
if len(seeds) < min_samples:
labels[p] = -1
continue
labels[p] = c
q = 0
while q < len(seeds):
i = int(seeds[q])
if labels[i] == -1:
labels[i] = c # 噪声点被核心点"收编"为边界点
elif labels[i] == -2:
labels[i] = c
nbrs = list(np.where(D[i] <= eps)[0])
if len(nbrs) >= min_samples:
```

```
seeds += [int(t) for t in nbrs if t not in seeds]
q += 1
c += 1
return labels
# ---------------------------------------------------------------------------- EMFS 集成（论文 13）
class EMFSEnsemble:
"""MFS + 局部单保真代理（local SFS）的自适应加权集成［论文 13 的 EMFS 思路］。
论文流程：① 用 HF+LF 建多保真代理 MFS；② 用 DBSCAN 找出 MFS 建不准的失真区；
③ 在失真区内**只用 HF** 建局部单保真代理 local SFS；④ 用自适应权重把两者融合。
> ⚠ 论文 13 的**权重公式原文未给出**（第 05 讲已标注"原文未获取"）。本实现采用
> 一个直接的工程方案：**按局部留一误差的倒数加权**——哪个模型在这一点附近历史上
> 更准，谁的权重就更大。这是我们的选择，不是论文数值。
"""
def __init__(self, gp_factory: Optional[Callable[[], object]] = None,
eps: float = 0.25, min_samples: int = 4, delta: float = 1e-6):
self.gp_factory = gp_factory if gp_factory is not None else (lambda: TinyKriging())
self.eps = float(eps)
self.min_samples = int(min_samples)
self.delta = float(delta)
def fit(self, X_lf: np.ndarray, y_lf: np.ndarray,
X_hf: np.ndarray, y_hf: np.ndarray) -> "EMFSEnsemble":
X_hf = np.atleast_2d(np.asarray(X_hf, float))
y_hf = np.asarray(y_hf, float).ravel()
self.X_hf, self.y_hf = X_hf, y_hf
# ① 多保真代理
self.mfs = CoKriging(gp_factory=self.gp_factory).fit(X_lf, y_lf, X_hf, y_hf)
# ② 失真区检测：在 HF 样本上做密度聚类
self.labels = dbscan(X_hf, eps=self.eps, min_samples=self.min_samples)
self.regions = [np.where(self.labels == c)[0]
for c in sorted(set(self.labels) - {-1})]
# ③ 每个失真区里只用 HF 建局部单保真代理
self.locals: List[object] = []
self.local_err: List[float] = []
for idx in self.regions:
if idx.size < 5: # 样本太少就建不了局部模型
self.locals.append(None)
self.local_err.append(np.inf)
continue
try:
m = self.gp_factory().fit(X_hf[idx], y_hf[idx])
err = self._loo_rmse(X_hf[idx], y_hf[idx])
except (ValueError, np.linalg.LinAlgError):
m, err = None, np.inf
self.locals.append(m)
self.local_err.append(err)
# 全局单保真代理（噪声点所在区域没有局部模型时的兜底）
try:
self.sfs = self.gp_factory().fit(X_hf, y_hf)
self.sfs_err = self._loo_rmse(X_hf, y_hf)
except (ValueError, np.linalg.LinAlgError):
self.sfs, self.sfs_err = None, np.inf
self.mfs_err = self._loo_rmse(X_hf, y_hf, model=self.mfs)
return self
@staticmethod
def _loo_rmse(X: np.ndarray, y: np.ndarray, model: Optional[object] = None,
gp_factory: Optional[Callable[[], object]] = None) -> float:
"""留一 RMSE（用于给各模型定权重）。"""
factory = gp_factory if gp_factory is not None else (lambda: TinyKriging())
errs = []
for i in range(X.shape[0]):
keep = np.ones(X.shape[0], dtype=bool)
keep[i] = False
try:
m = model if model is not None else None
# 有现成模型时无法"去掉一个样本"，故统一重新拟合
if m is None:
m = factory().fit(X[keep], y[keep])
pred = m.predict(X[i].reshape(1, -1))
errs.append(float(np.asarray(pred[0]).ravel()[0]) - float(y[i]))
except (ValueError, np.linalg.LinAlgError):
errs.append(float("nan"))
return float(np.sqrt(np.nanmean(np.square(errs))))
def _nearest_region(self, x: np.ndarray) -> int:
if not self.regions:
return -1
dists = [float(np.min(np.linalg.norm(self.X_hf[idx] - x, axis=1)))
for idx in self.regions]
return int(np.argmin(dists))
def predict(self, Xs: np.ndarray) -> Tuple[np.ndarray, np.ndarray]:
Xs = np.atleast_2d(np.asarray(Xs, float))
mu_mfs, sd_mfs = self.mfs.predict(Xs)
out_mu = np.empty(Xs.shape[0])
out_sd = np.empty(Xs.shape[0])
for i, x in enumerate(Xs):
k = self._nearest_region(x)
loc = self.locals[k] if k >= 0 else None
e_loc = self.local_err[k] if k >= 0 else np.inf
if loc is None: # 没局部模型 → 用全局 SFS
loc, e_loc = self.sfs, self.sfs_err
if loc is None: # 连全局 SFS 都建不起来 → 退回 MFS
out_mu[i], out_sd[i] = mu_mfs[i], sd_mfs[i]
continue
```

```
mu_sfs, sd_sfs = loc.predict(x.reshape(1, -1))
mu_sfs = float(np.asarray(mu_sfs).ravel()[0])
sd_sfs = float(np.asarray(sd_sfs).ravel()[0])
# ④ 自适应权重：误差越小权重越大（本代码库的工程选择，非论文公式）
w_mfs = 1.0 / (self.mfs_err + self.delta)
w_sfs = 1.0 / (e_loc + self.delta)
s = w_mfs + w_sfs
w_mfs, w_sfs = w_mfs / s, w_sfs / s
out_mu[i] = w_mfs * float(mu_mfs[i]) + w_sfs * mu_sfs
out_sd[i] = math.sqrt(max(w_mfs * float(sd_mfs[i]) ** 2
+ w_sfs * sd_sfs ** 2, 0.0))
return out_mu, out_sd
# ---------------------------------------------------------------------------- 通用组合选择
def hedge_choose(gains: Sequence[float], eta: float = 0.5,
rng: Optional[np.random.Generator] = None) -> int:
"""按累积收益的 softmax 概率选择一个候选（多采集函数"组合策略"的通用做法）。
p_k ∝ exp(η · g_k)
说明：本全集的第 02 讲用**多个不同 g 的 GEI 并行加点**，但并未给出如何在多个采集
准则之间做选择的公式；这里的 softmax 对冲是优化领域的通用技巧，**不对应某篇具体
论文**，请勿当作原文方法引用。
"""
g = np.asarray(gains, float).ravel()
if g.size == 0:
raise ValueError("gains 不能为空")
if eta <= 0:
raise ValueError("eta 必须为正")
z = eta * (g - np.max(g))
p = np.exp(z)
p = p / np.sum(p)
if rng is None:
return int(np.argmax(p))
return int(rng.choice(len(p), p=p))
# ---------------------------------------------------------------------------- 自检
def _self_test() -> None:
rng = np.random.default_rng(20240905)
# ---- 1) Filter-GEI 的自适应权重与阈值（论文 02 式 10–12）
w, T = filter_gei_threshold(0.0, 1.0, v_hf_min=2.0, v_hf_mean=5.0)
assert abs(w - 1.0) < 1e-12 and abs(T - 2.0) < 1e-12, "σ² DF=0 时 ω、T 取值错误"
w, T = filter_gei_threshold(1.0, 1.0, 2.0, 5.0)
assert abs(w - 0.5) < 1e-12 and abs(T - 3.5) < 1e-12, "σ² DF=σ² LF 时 ω 应为 0.5"
w2, _ = filter_gei_threshold(100.0, 1.0, 2.0, 5.0)
assert w2 < 0.1, "LF 与 HF 相关性差时 ω 应趋近 0"
# 单调性：σ² DF 越大（相关性越差），ω 越小、T 越靠近历史均值
ws = [filter_gei_threshold(s, 1.0, 2.0, 5.0) for s in (0.0, 0.25, 1.0, 4.0, 25.0)]
assert all(ws[i][0] > ws[i + 1][0] for i in range(len(ws) - 1)), "ω 应随 σ² DF 单调下降"
assert all(ws[i][1] < ws[i + 1][1] for i in range(len(ws) - 1)), "T 应随 σ²_DF 单调上升"
# ---- 2) 层次聚类去重叠：合并"距离最近的两类"，直到最小类间距离 > d_T
X = np.array([[0.0], [0.01], [1.0], [5.0]])
cl = hierarchical_decluster(X, d_T=0.1)
# 只有 0 与 1（距离 0.01 ⩽ 0.1）被合并；此后最小类间距离 0.99 > 0.1，停止
assert len(cl) == 3, f"应聚成 3 类，实为 {len(cl)}：{cl}"
assert sorted(sum(cl, [])) == [0, 1, 2, 3], "聚类结果未覆盖全部点"
assert [0, 1] in cl, "最近的两点应被合并"
assert len(hierarchical_decluster(X, d_T=10.0)) == 1, "阈值极大时应聚成 1 类"
assert len(hierarchical_decluster(X, d_T=0.0)) == 4, "阈值 0 时只有重合点才会被合并"
# ---- 3) 分流与成本折算
pred = np.array([1.0, 3.0, 5.0])
hf_idx, lf_idx = fidelity_split(pred, T=3.5)
assert list(hf_idx) == [0, 1] and list(lf_idx) == [2], "分流结果错误"
rep = budget_report(10, 50, cost_ratio=0.1)
assert abs(rep["equivalent_hf"] - 15.0) < 1e-12, "等效 HF 次数错误"
# ---- 4) DBSCAN：两簇 + 一个噪声点
Xd = np.vstack([rng.normal(0.0, 0.05, size=(20, 2)),
rng.normal(5.0, 0.05, size=(20, 2)),
np.array([[20.0, 20.0]])])
lab = dbscan(Xd, eps=0.5, min_samples=4)
assert len(set(lab) - {-1}) == 2, f"应识别出 2 个簇，实为 {set(lab)}"
assert lab[-1] == -1, "孤立点应被标为噪声"
assert np.all(lab[:20] == lab[0]) and np.all(lab[20:40] == lab[20]), "簇内标签不一致"
assert lab[0] != lab[20], "两簇标签不应相同"
# ---- 5) Co-Kriging：应当优于"只用少量 HF"的单保真模型
def lf(x: np.ndarray) -> np.ndarray:
x = np.atleast_2d(np.asarray(x, float))
return (np.sin(2.0 * np.pi * x[:, 0]) + 0.5 * x[:, 1]).ravel()
def hf(x: np.ndarray) -> np.ndarray:
x = np.atleast_2d(np.asarray(x, float))
return (1.6 * (np.sin(2.0 * np.pi * x[:, 0]) + 0.5 * x[:, 1])
+ 0.8 * (x[:, 0] - 0.5) ** 2).ravel()
Xlf = rng.random((40, 2))
ylf = lf(Xlf)
Xhf = rng.random((6, 2))
yhf = hf(Xhf)
ck = CoKriging().fit(Xlf, ylf, Xhf, yhf)
Xt = rng.random((300, 2))
yt = hf(Xt)
mu_ck, sd_ck = ck.predict(Xt)
r2_ck = 1.0 - np.sum((yt - mu_ck) ** 2) / np.sum((yt - np.mean(yt)) ** 2)
only_hf = _TinyKriging().fit(Xhf, yhf)
mu_hf, _ = only_hf.predict(Xt)
r2_hf = 1.0 - np.sum((yt - mu_hf) ** 2) / np.sum((yt - np.mean(yt)) ** 2)
assert r2_ck > r2_hf, f"Co-Kriging({r2_ck:.3f}) 未优于仅用 HF({r2_hf:.3f})"
assert r2_ck > 0.8, f"Co-Kriging 精度不足：{r2_ck:.3f}"
```

```
assert 0.8 < ck.rho < 2.5, f"ρ 估计偏离真值 1.6 太远：{ck.rho}"
assert np.all(sd_ck >= 0), "预测标准差出现负值"
# 训练点处应接近插值
mu_at_hf, _ = ck.predict(Xhf)
assert np.max(np.abs(mu_at_hf - yhf)) < 0.35 * (yhf.max() - yhf.min() + TINY), \
"Co-Kriging 在 HF 训练点处偏离过大"
# ---- 6) EMFS 集成：能跑通，且权重确实偏向误差更小的模型 ----
em = EMFSEnsemble(eps=0.9, min_samples=4).fit(Xlf, ylf, Xhf, yhf)
mu_em, sd_em = em.predict(Xt)
assert np.all(np.isfinite(mu_em)) and np.all(sd_em >= 0), "EMFS 预测出现异常值"
r2_em = 1.0 - np.sum((yt - mu_em) ** 2) / np.sum((yt - np.mean(yt)) ** 2)
assert r2_em > 0.5, f"EMFS 精度过低：{r2_em:.3f}"
# ---- 7) softmax 对冲：收益越高被选中概率越大 ----
p = np.array([hedge_choose([0.0, 0.0, 0.0], eta=1.0,
rng=np.random.default_rng(s)) for s in range(300)])
assert set(np.unique(p).tolist()) == {0, 1, 2}, "收益相同时各策略都应可能被选中"
q = np.array([hedge_choose([0.0, 5.0, 0.0], eta=1.0,
rng=np.random.default_rng(s)) for s in range(200)])
assert np.mean(q == 1) > 0.9, "高收益策略应被大概率选中"
# ---- 8) 异常输入 ----
for bad in [lambda: filter_gei_threshold(1.0, 0.0, 1.0, 2.0),
lambda: filter_gei_threshold(-1.0, 1.0, 1.0, 2.0),
lambda: hierarchical_decluster(np.zeros((3, 2)), d_T=-1.0),
lambda: dbscan(np.zeros((3, 2)), eps=0.0),
lambda: dbscan(np.zeros((3, 2)), eps=0.5, min_samples=0),
lambda: budget_report(1, 1, cost_ratio=0.0),
lambda: hedge_choose([], eta=1.0),
lambda: CoKriging().fit(np.zeros((2, 2)), np.zeros(2),
np.zeros((2, 3)), np.zeros(2))]:
try:
bad()
except ValueError:
pass
else:
raise AssertionError("非法输入未报错")
print(f"[multifidelity.py] 自检通过：ω/T单调 / 层次聚类 / DBSCAN(2簇+噪声) / "
f"CoKriging(R²={r2_ck:.3f} vs 仅HF {r2_hf:.3f}, ρ={ck.rho:.2f}) / "
f"EMFS(R²={r2_em:.3f}) / 对冲选择")
if __name__ == "__main__":
_self_test()
```

code/optimizer.py

```
"""optimizer.py —— 昂贵黑箱优化器：DE / PSO / EGO / CR-EI / 代理辅助 DE
本模块把本全集讨论过的几类"搜索策略"实现成可直接调用的函数，方便对照：
| 函数 | 对应论文 | 要点 |
|---|---|---|
| `differential_evolution` | 论文 07（GSDE 的底座） | best/1 与 rand/1 变异、二项/指数交叉 |
| `pso` | 论文 07（GSDE 用 PSO 在局部 RBF 上找预测父点） | 标准惯性权重 + 个体/群体最优 |
| `ego` | 论文 01/02/04/11 的公共底座 | 代理 + 期望改善（EI）的序贯加点 |
| `calibrated_ei` / `recalibrated_ei` / `cr_ei_step` | **论文 01（CR-EI）** | 抬高原 incumbent 治过度探索、互补分区治过度开发 |
| `surrogate_assisted_de` | **论文 07（GSDE）** | 代理**直接参与生成子代基因**（紧耦合），而非只做筛选 |
关于 CR-EI 的一句话原理（第 01 讲）：
EI 的 incumbent 固定为当前最优 y_PBS 时，建议点会越来越"往远处瞎逛"。
CR-EI 不动 σ，而是**改 incumbent ξ**：把 ξ 抬高到 ξ_Calibrated 得到 CEI（收紧到近处
开发），并用与 CEI 采样区**互补**的 REI 保持全局探索；三者由一个"突破计数器 + 中位数
阈值"调度切换［原文 P01 式 17–18］。
代理模型的**鸭子接口**（本模块不 import 任何具体模型，避免耦合）：
obj = factory() # 工厂，每次调用返回新模型
obj.fit(X, y) # X:(n,d) 原始尺度，y:(n,)
obj.predict(Xs) -> (mean, sd) # 两个数组，形状均为 (m,)
`code/gp.py` 的 `Kriging.predict` 正好符合；`code/rbf.py` 的 RBF 只有均值，需要包一层。
**只依赖 numpy**。运行自检：
python code/optimizer.py
"""
from __future__ import annotations
import math
from typing import Callable, Dict, List, Optional, Sequence, Tuple
import numpy as np
__all__ = [
"differential_evolution", "pso", "expected_improvement",
"calibrated_ei", "recalibrated_ei", "cr_ei_step",
"ego", "surrogate_assisted_de",
]
_TINY = 1e-12
_ERF = np.vectorize(math.erf, otypes=[float])
# ---------------------------------------------------------------------- 公共工具
def _norm_pdf(u: np.ndarray) -> np.ndarray:
return np.exp(-0.5 * np.asarray(u, float) ** 2) / math.sqrt(2.0 * math.pi)
def _norm_cdf(u: np.ndarray) -> np.ndarray:
erf = _ERF
return 0.5 * (1.0 + erf(np.asarray(u, float) / math.sqrt(2.0)))
def expected_improvement(mean: np.ndarray, sd: np.ndarray,
incumbent: float) -> np.ndarray:
"""标准 EI：EI = (ξ − μ)·Φ(u) + s·φ(u)，u = (ξ − μ)/s。"""
mean = np.asarray(mean, float)
sd = np.asarray(sd, float)
imp = float(incumbent) - mean
out = np.zeros_like(mean)
```

```
pos = sd > TINY
u = np.zeros_like(mean)
u[pos] = imp[pos] / sd[pos]
out[pos] = imp[pos] * norm.cdf(u[pos]) + sd[pos] * norm.pdf(u[pos])
return out
# ---------------------------------------------------------------------- 差分进化
def differential_evolution(func: Callable[[np.ndarray], float],
bounds: Sequence[Tuple[float, float]],
pop_size: int = 50,
F: float = 0.7,
CR: float = 0.9,
max_nfe: int = 1000,
strategy: str = "best/1",
crossover: str = "bin",
seed: int = 0,
return_history: bool = False):
"""差分进化（DE）。
strategy : 'best/1'（开发强）或 'rand/1'（探索强）
crossover : 'bin'（二项交叉）或 'exp'（指数交叉，GSDE 用的是指数交叉）
"""
rng = np.random.default_rng(seed)
bounds = np.asarray(bounds, float)
lo, hi = bounds[:, 0], bounds[:, 1]
d = bounds.shape[0]
if pop_size < 4:
raise ValueError("pop_size 至少为 4（DE 需要 3 个不同的随机个体）")
P = lo + rng.random((pop_size, d)) * (hi - lo)
f = np.array([float(func(x)) for x in P])
nfe = pop_size
best_idx = int(np.argmin(f))
hist: List[float] = [float(f[best_idx])]
while nfe < max_nfe:
for i in range(pop_size):
if nfe >= max_nfe:
break
# --- 变异
idxs = [j for j in range(pop_size) if j != i]
r1, r2, r3 = rng.choice(idxs, size=3, replace=False)
if strategy == "best/1":
base = P[best_idx] # v = x_best + F·(x_r1 − x_r2)
elif strategy == "rand/1":
base = P[r3] # v = x_r3 + F·(x_r1 − x_r2)
else:
raise ValueError(f"未知策略：{strategy}")
v = np.clip(base + F * (P[r1] - P[r2]), lo, hi)
# --- 交叉
u = P[i].copy()
if crossover == "bin":
irand = int(rng.integers(0, d))
mask = rng.random(d) < CR
mask[irand] = True # 至少一维来自变异向量
u = np.where(mask, v, u)
elif crossover == "exp":
k = int(rng.integers(0, d))
L = 0
while rng.random() < CR and L < d:
u[k] = v[k]
k = (k + 1) % d
L += 1
else:
raise ValueError(f"未知交叉方式：{crossover}")
fu = float(func(u))
nfe += 1
if fu <= f[i]: # 贪婪替换
P[i], f[i] = u, fu
if fu < f[best_idx]:
best_idx = i
hist.append(float(f[best_idx]))
if return_history:
return P[best_idx], float(f[best_idx]), np.array(hist)
return P[best_idx], float(f[best_idx])
# ---------------------------------------------------------------------- 粒子群
def pso(func: Callable[[np.ndarray], float],
bounds: Sequence[Tuple[float, float]],
n_particles: int = 50,
n_iter: int = 100,
w: float = 0.72,
c1: float = 1.49,
c2: float = 1.49,
seed: int = 0,
return_history: bool = False):
"""标准粒子群优化（PSO）。
论文 07（GSDE）在**局部 RBF 代理**上用 PSO（种群 50、迭代 100 次）寻找"预测父点"
x_i*，因此这里的 PSO 既可直接用于真实函数，也可用于代理模型。
"""
rng = np.random.default_rng(seed)
bounds = np.asarray(bounds, float)
lo, hi = bounds[:, 0], bounds[:, 1]
d = bounds.shape[0]
X = lo + rng.random((n_particles, d)) * (hi - lo)
V = rng.uniform(-1.0, 1.0, size=(n_particles, d)) * (hi - lo) * 0.1
f = np.array([float(func(x)) for x in X])
```

```
pbest_x, pbest_f = X.copy(), f.copy()
gbest_i = int(np.argmin(pbest_f))
gbest_x, gbest_f = pbest_x[gbest_i].copy(), float(pbest_f[gbest_i])
hist = [gbest_f]

for _ in range(n_iter):
r1 = rng.random((n_particles, d))
r2 = rng.random((n_particles, d))
V = w * V + c1 * r1 * (pbest_x - X) + c2 * r2 * (gbest_x - X)
X = np.clip(X + V, lo, hi)
f = np.array([float(func(x)) for x in X])
improved = f < pbest_f
pbest_x[improved] = X[improved]
pbest_f[improved] = f[improved]
i = int(np.argmin(pbest_f))
if pbest_f[i] < gbest_f:
gbest_f, gbest_x = float(pbest_f[i]), pbest_x[i].copy()
hist.append(gbest_f)

if return_history:
return gbest_x, gbest_f, np.array(hist)
return gbest_x, gbest_f

# ---------------------------------------------------------------------------- CR-EI（论文 01）
def calibrated_ei(mean: np.ndarray, sd: np.ndarray, xi_calibrated: float) -> np.ndarray:
"""CEI：把 incumbent 抬到 ξ_Calibrated（> y_PBS）后的 EI。
抬高 incumbent 会让原本 z ≤ 0 的点变成 z > 0，从而把采样区重新约束在
ŷ(x*) < ξ_Calibrated 之内，抑制"过度探索"。
"""
return expected_improvement(mean, sd, xi_calibrated)

def recalibrated_ei(mean: np.ndarray, sd: np.ndarray, xi_calibrated: float,
X: np.ndarray, x_pbs: np.ndarray,
d_T: float, eps: float = 1e-3) -> np.ndarray:
"""REI：CEI 乘以**互补**分区掩码 δ₂ —— 只在 x_PBS 的 d_T 邻域**之外**取点。
δ₁ = 1 当 ε < ‖x − x_PBS‖ ≤ d_T（近处，归 CEI）
δ₂ = 1 − δ₁ （远处，归 REI）
三者合起来覆盖整个设计空间，这是 CR-EI 同时抗过度探索与过度开发的关键。
"""
dist = np.linalg.norm(np.asarray(X, float) - np.asarray(x_pbs, float).ravel(), axis=1)
d1 = ((dist > eps) & (dist <= d_T)).astype(float)
return calibrated_ei(mean, sd, xi_calibrated) * (1.0 - d1)

def cr_ei_step(mean: np.ndarray, sd: np.ndarray, X: np.ndarray,
y: np.ndarray, x_pbs: np.ndarray,
d_T: float, beta: float = 0.1,
n_breakthrough: int = 0, eps: float = 1e-3) -> Dict[str, object]:
"""CR-EI 一步决策［原文 P01 式 17–18］。
参数
----
mean, sd : 候选点上的代理预测
X, y : 已评估样本（用于取中位数阈值 y_T 与当前最优 y_PBS）
x_pbs : 当前最优样本坐标
d_T : 距离阈值，划分"近处/远处"
beta : incumbent 参数，原文扫描后取 β=0.1 综合最优
n_breakthrough : 突破计数器（>0 表示 EI 仍在取得进展）
返回 dict：{'mode': ..., 'indices': [...]}，indices 是本轮建议评估的候选点下标
（最多 2 个，对应原论文的"EI + REI 并行"）。
"""
X = np.asarray(X, float)
y = np.asarray(y, float).ravel()
if X.shape[0] != y.size:
raise ValueError("X 与 y 样本数不一致")
if not 0.0 < beta < 1.0:
raise ValueError("beta 应落在 (0,1)")

y_pbs = float(np.min(y))
y_T = float(np.median(y))
# ξ_Calibrated：把 incumbent 从 y_PBS 抬高，抬升幅度按 |y| 量级缩放
scale = max(abs(y_pbs), abs(y_T), _TINY)
xi_calibrated = y_pbs + beta * abs(y_T - y_pbs) if abs(y_T - y_pbs) > _TINY \
else y_pbs + beta * scale

ei = expected_improvement(mean, sd, y_pbs)
i_ei = int(np.argmax(ei))

if n_breakthrough == 0:
return {"mode": "EI", "indices": [i_ei],
"xi_calibrated": xi_calibrated, "y_T": y_T}

cei = calibrated_ei(mean, sd, xi_calibrated)
dist = np.linalg.norm(X - np.asarray(x_pbs, float).ravel(), axis=1)
d1 = ((dist > eps) & (dist <= d_T)).astype(float)
i_cei = int(np.argmax(cei * d1)) if np.any(d1 > 0) else i_ei

rei = recalibrated_ei(mean, sd, xi_calibrated, X, x_pbs, d_T, eps)
i_rei = int(np.argmax(rei))

if float(y[i_ei]) < y_T or n_breakthrough > 0 and float(ei[i_ei]) > 0:
# EI 有进展：继续用 EI，同时派 REI 去远处
mode = "EI+REI"
idxs = [i_ei, i_rei]
else:
# EI 疑似过度探索：换 CEI（近处开发）+ REI（远处探索）
mode = "CEI+REI"
idxs = [i_cei, i_rei]

out: List[int] = []
for i in idxs:
if i not in out:
out.append(i)
return {"mode": mode, "indices": out, "xi_calibrated": xi_calibrated, "y_T": y_T}
```

```
# ---------------------------------------------------------------------------- EGO 主循环
def ego(func: Callable[[np.ndarray], float],
bounds: Sequence[Tuple[float, float]],
surrogate_factory: Callable[[], object],
n_init: int = 20,
n_iter: int = 30,
n_candidates: int = 2048,
seed: int = 0,
return_history: bool = False):
"""贝叶斯优化的标准 EGO 循环：初始采样 → 循环 {拟合代理 → 最大化 EI → 真实评估}。
surrogate_factory 是**工厂函数**，每轮返回一个新模型（鸭子接口见模块文档）。
"""
rng = np.random.default_rng(seed)
bounds = np.asarray(bounds, float)
lo, span = bounds[:, 0], bounds[:, 1] - bounds[:, 0]
d = bounds.shape[0]
X = lo + rng.random((n_init, d)) * span
y = np.array([float(func(x)) for x in X])
hist = [float(np.min(y))]
for _ in range(n_iter):
mdl = surrogate_factory().fit(X, y)
Xc = lo + rng.random((n_candidates, d)) * span
mean, sd = mdl.predict(Xc)
ei = expected_improvement(mean, sd, float(np.min(y)))
x_new = Xc[int(np.argmax(ei))]
y_new = float(func(x_new))
X = np.vstack([X, x_new])
y = np.append(y, y_new)
hist.append(float(np.min(y)))
i = int(np.argmin(y))
if return_history:
return X[i], float(y[i]), np.array(hist)
return X[i], float(y[i])
# ---------------------------------------------------------------------------- GSDE（论文 07）
def surrogate_assisted_de(func: Callable[[np.ndarray], float],
bounds: Sequence[Tuple[float, float]],
surrogate_factory: Callable[[], object],
pop_size: int = 50,
F: float = 0.7,
CR: float = 0.9,
top_T: int = 9,
pso_particles: int = 50,
pso_iters: int = 100,
local_k: Optional[int] = None,
local_radius: float = 0.25,
max_nfe: int = 300,
seed: int = 0,
return_history: bool = False):
"""代理辅助差分进化——GSDE 的骨架（论文 07）。
与"松散耦合"的差别：代理模型**直接参与生成子代基因**。流程为
1. 对每个父代 x_i，取它的 k 个最近邻样本建**局部代理**（GSDE 用局部 RBF）；
2. 用 PSO 在这个局部代理上、于 x_i 的邻域内找"预测父点" x_i*；
3. best/1 变异得到 v_i，再用**指数交叉**把 v_i 与 x_i* 缝合成候选 u_i；
4. 按**父代适应度**给候选排序，只挑前 T 个做真实评估（原文 T=9）并替换最差的 T 个。
> 原文的关键工程动机：每轮若把全部候选都送去做 CFD，开销不可承受；而"优秀父代
> 产生的子代更可能携带好基因"，所以只评估最好的 T 个「原文 P07 §Proposed algorithm」。
>
> **实现说明**：原文未公开 k（邻居数）与局部邻域半径的取值，本实现的
> `local_k` 与 `local_radius` 是我们自己的工程选择，不是论文数值。
"""
rng = np.random.default_rng(seed)
bounds = np.asarray(bounds, float)
lo, hi = bounds[:, 0], bounds[:, 1]
d = bounds.shape[0]
lo_arr, span_arr = lo, hi - lo
if top_T < 1 or top_T > pop_size:
raise ValueError("top_T 应落在 [1, pop_size]")
P = lo + rng.random((pop_size, d)) * span_arr
f = np.array([float(func(x)) for x in P])
nfe = pop_size
hist = [float(np.min(f))]
def unit_to_raw(Z: np.ndarray) -> np.ndarray:
return lo_arr + np.asarray(Z, float) * span_arr
while nfe < max_nfe:
# 0. 归一化坐标（代理一律在 [0,1]^d 上工作）
Z = (P - lo_arr) / span_arr
k_nb = int(local_k) if local_k else min(pop_size, max(d + 2, 3 * d + 6))
# 1~2. 逐父代建**局部**代理，并用 PSO 在其邻域内找"预测父点" x_i*
x_stars = np.empty_like(P)
for i in range(pop_size):
dist = np.linalg.norm(Z - Z[i], axis=1)
nb = np.argsort(dist)[:k_nb]
try:
mdl = surrogate_factory().fit(Z[nb], f[nb])
except ValueError:
mdl = None
if mdl is None:
x_stars[i] = P[i]
continue
def neg_pred(z: np.ndarray, _mdl=mdl) -> float:
z = np.atleast_2d(np.asarray(z, float))
```

```
val = mdl.predict(z)
mean = val[0] if isinstance(val, tuple) else val
return float(np.asarray(mean).ravel()[0])
# 局部邻域：以父代为中心、半径 local_radius（归一化尺度），并被设计域截断
zc = Z[i]
box = [(max(0.0, zc[i] - local_radius), min(1.0, zc[i] + local_radius))
for i in range(d)]
zbest, fbest_sur, _ = pso(neg_pred, box, n_particles=pso_particles,
n_iter=pso_iters, seed=int(rng.integers(1 << 30)))
# 代理的建议若不如父代本身，就退回父代（避免代理误导 —— 即"负迁移"的同类风险）
x_stars[i] = P[i] if float(fbest_sur) >= float(f[i]) else unit_to_raw(zbest)
# 3. best/1 变异 + 与预测父点的指数交叉
i_best = int(np.argmin(f))
cand, parent_idx = [], []
for i in range(pop_size):
idxs = [j for j in range(pop_size) if j != i]
r1, r2 = rng.choice(idxs, size=2, replace=False)
v = np.clip(P[i_best] + F * (P[r1] - P[r2]), lo, hi)
u = v.copy()
k = int(rng.integers(0, d))
L = 0
while rng.random() < CR and L < d:
u[k] = x_stars[i][k] # 代理的"好基因"直接进入子代
k = (k + 1) % d
L += 1
cand.append(np.clip(u, lo, hi))
parent_idx.append(i)
cand = np.array(cand)
# 4. 按父代适应度排序，只评估前 T 个
order = np.argsort(f[np.array(parent_idx)])
pick = order[:top_T]
for i in pick:
fu = float(func(cand[i]))
nfe += 1
i = parent_idx[i]
if fu < f[i]:
P[i], f[i] = cand[i], fu
hist.append(float(np.min(f)))
if nfe >= max_nfe:
break
i = int(np.argmin(f))
if return_history:
return P[i], float(f[i]), np.array(hist)
return P[i], float(f[i])
# ---------------------------------------------------------------------------- 自检
class DemoSurrogate:
"""自检用的轻量代理（三次 RBF + 线性尾项，sd 用"到最近样本的距离"近似）。
放在自检里而不是模块顶层，是为了保持 `optimizer.py` 与具体代理模型解耦。
"""
def __init__(self):
self.X = None
self.v = None
self.nn_scale = 1.0
def fit(self, X: np.ndarray, v: np.ndarray) -> "DemoSurrogate":
X = np.asarray(X, float)
v = np.asarray(v, float).ravel()
self.X, self.v = X, v
diff = X[:, None, :] - X[None, :, :]
D = np.sqrt(np.einsum("nmk,nmk->nm", diff, diff))
Phi = D ** 3
Pd = np.hstack([np.ones((X.shape[0], 1)), X])
m = Pd.shape[1]
A = np.zeros((X.shape[0] + m, X.shape[0] + m))
A[:X.shape[0], :X.shape[0]] = Phi
A[:X.shape[0], X.shape[0]:] = Pd
A[X.shape[0]:, :X.shape[0]] = Pd.T
self.sol = np.linalg.lstsq(A, np.concatenate([v, np.zeros(m)]), rcond=None)[0]
self.d = X.shape[1]
off = D[~np.eye(X.shape[0], dtype=bool)]
self.nn_scale = float(np.mean(off)) if off.size else 1.0
return self
def predict(self, Xs: np.ndarray) -> Tuple[np.ndarray, np.ndarray]:
Xs = np.atleast_2d(np.asarray(Xs, float))
diff = Xs[:, None, :] - self.X[None, :, :]
D = np.sqrt(np.einsum("nmk,nmk->nm", diff, diff))
Phi = D ** 3
Pd = np.hstack([np.ones((Xs.shape[0], 1)), Xs])
mean = Phi @ self.sol[:self.X.shape[0]] + Pd @ self.sol[self.X.shape[0]:]
nn = D.min(axis=1)
sd = np.clip(nn / max(self.nn_scale, TINY), 1e-6, 1.0) * (np.std(self.v) + 1e-9)
return mean, sd
def _self_test() -> None:
# ---- 目标函数（2 维 Branin，全局最优 0.397887）
def branin(x: np.ndarray) -> float:
x = np.atleast_1d(np.asarray(x, float))
x1, x2 = x[0], x[1]
a, b, c, r, s, t = 1.0, 5.1 / (4 * np.pi ** 2), 5 / np.pi, 6.0, 10.0, 1 / (8 * np.pi)
return float(a * (x2 - b * x1 ** 2 + c * x1 - r) ** 2 + s * (1 - t) * np.cos(x1) + s)
b2 = [(-5.0, 10.0), (0.0, 15.0)]
# 1) DE：两种策略、两种交叉都能收敛到接近全局最优
for strat in ("best/1", "rand/1"):
for cx in ("bin", "exp"):
```

```
xb, fb, hist = differential_evolution(branin, b2, pop_size=30, max_nfe=1500,
strategy=strat, crossover=cx,
seed=3, return_history=True)
assert fb < 0.5, f"DE {strat}/{cx} 未收敛：{fb}"
assert hist[-1] <= hist[0] + 1e-12, "最优值历史应单调不增"
# 2) PSO：能找到合理解
xp, fp, hp = pso(branin, b2, n_particles=40, n_iter=200, seed=5, return_history=True)
assert fp < 0.6, f"PSO 未收敛：{fp}"
assert hp[-1] <= hp[0] + 1e-12, "PSO 历史应单调不增"
# 3) EI 的数学性质
mean = np.array([0.0, 1.0, 2.0])
sd = np.array([1.0, 1.0, 1.0])
ei = expected_improvement(mean, sd, incumbent=0.0)
assert np.all(ei >= -1e-12), "EI 应为非负"
# 最小化问题：预测均值越低（看似越好），EI 越大
assert ei[0] > ei[1] > ei[2], "均值越低 EI 应越大"
assert expected_improvement(np.array([0.0]), np.array([0.0]), 0.0)[0] == 0.0
# 方差越大（越不确定）EI 越大
assert expected_improvement(np.array([1.0]), np.array([2.0]), 0.0)[0] > \
expected_improvement(np.array([1.0]), np.array([0.5]), 0.0)[0]
# 4) CEI / REI 的分区互补性（论文 01 的核心结构）
rng = np.random.default_rng(0)
X = rng.random((200, 2))
x_pbs = np.array([0.5, 0.5])
d_T = 0.3
mean = np.zeros(200)
sd = np.ones(200)
cei = calibrated_ei(mean, sd, xi_calibrated=1.0)
rei = recalibrated_ei(mean, sd, 1.0, X, x_pbs, d_T)
near = np.linalg.norm(X - x_pbs, axis=1) <= d_T
assert np.all(rei[near] == 0.0), "REI 不应在近处取点"
assert np.any(rei[~near] > 0.0), "REI 应在远处有值"
# 互补性：CEI · δ₁ + REI ≡ CEI（近处归 CEI，远处归 REI，合起来覆盖全空间）
dist_all = np.linalg.norm(X - x_pbs, axis=1)
delta1 = ((dist_all > 1e-3) & (dist_all <= d_T)).astype(float)
assert np.allclose(cei * delta1 + rei, cei), "CEI 与 REI 的分区不互补"
# 5) CR-EI 的三种调度模式都能被触发，且返回合法下标
v = np.linspace(-3.0, 3.0, 200)
res0 = cr_ei_step(mean, sd, X, v, x_pbs, d_T, beta=0.1, n_breakthrough=0)
assert res0["mode"] == "EI" and len(res0["indices"]) == 1
res1 = cr_ei_step(mean, sd, X, v, x_pbs, d_T, beta=0.1, n_breakthrough=5)
assert res1["mode"] in ("EI+REI", "CEI+REI"), res1["mode"]
assert len(res1["indices"]) <= 2 and all(0 <= i < 200 for i in res1["indices"])
# ξ_Calibrated 必须高于当前最优（这是 CEI 的定义）
assert res1["xi_calibrated"] > float(np.min(v))
# beta 越大，抬升幅度越大
a = cr_ei_step(mean, sd, X, v, x_pbs, d_T, beta=0.1, n_breakthrough=1)["xi_calibrated"]
bb = cr_ei_step(mean, sd, X, v, x_pbs, d_T, beta=0.5, n_breakthrough=1)["xi_calibrated"]
assert bb > a, "β 越大 incumbent 抬升应越多"
# 6) EGO：在 Branin 上，30 轮加点后应显著优于初始随机采样
xg, fg, hg = ego(branin, b2,  DemoSurrogate, n_init=15, n_iter=40,
n_candidates=1500, seed=11, return_history=True)
assert fg < 0.45, f"EGO 未收敛：{fg}"
assert hg[-1] <= hg[0] + 1e-12, "EGO 历史应单调不增"
assert hg[-1] < hg[0], "EGO 相对初始采样应有改进"
# 7) GSDE 骨架：同样的真实评估预算（240 次）下，应优于纯随机搜索。
# 随机搜索单次结果波动很大，故取 3 个随机种子的平均最优值做基线。
xs, fs, hs = surrogate_assisted_de(branin, b2,  DemoSurrogate,
pop_size=20, top_T=5,
pso_particles=10, pso_iters=20,
max_nfe=240, seed=13, return_history=True)
lo = np.array([b[0] for b in b2])
sp = np.array([b[1] - b[0] for b in b2])
rand_bests = []
for rs in (99, 7, 11):
r = np.random.default_rng(rs)
rand_bests.append(min(float(branin(lo + r.random(2) * sp)) for  in range(240)))
rand_mean = float(np.mean(rand_bests))
assert fs < rand_mean, f"GSDE({fs:.4f}) 未优于随机搜索均值({rand_mean:.4f})"
assert hs[-1] <= hs[0] + 1e-12, "GSDE 历史应单调不增"
# 8) 异常输入
for bad in [lambda: differential_evolution(branin, b2, pop_size=3),
lambda: differential_evolution(branin, b2, strategy="cur/1"),
lambda: differential_evolution(branin, b2, crossover="weird"),
lambda: surrogate_assisted_de(branin, b2,  DemoSurrogate,
pop_size=5, top_T=9),
lambda: cr_ei_step(mean, sd, X, v[:10], x_pbs, d_T),
lambda: cr_ei_step(mean, sd, X, v, x_pbs, d_T, beta=1.5)]:
try:
bad()
except ValueError:
pass
else:
raise AssertionError("非法输入未报错")
print(f"[optimizer.py] 自检通过：DE(4 组合)/PSO({fp:.4f})/EI性质/"
f"CR-EI三模式/EGO({fg:.4f})/GSDE({fs:.4f} vs 随机均值 {rand_mean:.4f})")
if  name  == "  main  ":
_self_test()
```

code/rbf.py

```
"""rbf.py —— 径向基函数（RBF）代理模型

为什么单独给 RBF 一个模块：本全集第 03 讲（**论文 07，GSDE**）在 126 维叶栅优化里
**明确放弃了 Kriging 而选择 RBF**，理由是"高维设计空间中 RBF 的预测精度强于其他常见
代理，且训练时间随变量数增长仍可接受"［原文 P07 §Proposed algorithm］。GSDE 使用的是

ŷ(x) = Σ_i λ_i · φ( ‖ x − x_i ‖ ) + c₀ + Σ_j c_j · x_j   （φ(r)=r³，三次 RBF + 线性尾项）

即**三次径向基 + 线性多项式尾项**。这个结构有两个必须知道的性质：

1. **插值性**：只要样本点互不相同，线性系统满秩，模型必定精确通过所有训练点；
2. **线性再生性**：带线性尾项后，若真实函数是一次多项式，模型可以**精确复现**。

代价是：RBF 不自带不确定性估计（不像 Kriging 有 σ），所以它更适合"进化算法里当
局部加速器"，而不适合直接拿去做 EI 类采集函数——这正是 GSDE 把它嵌进 DE、而不是
拿它做贝叶斯优化的原因。

**只依赖 numpy**。运行自检：
python code/rbf.py
自检内容：插值性、线性再生性、留一误差、边界外推行为、异常输入。
"""

from __future__ import annotations

from typing import Callable, Optional, Sequence, Tuple

import numpy as np

__all__ = ["RBFInterpolant", "cubic_kernel", "linear_kernel",
"thin_plate_kernel", "gaussian_kernel", "fit_rbf", "leave_one_out_rmse"]

_TINY = 1e-12

# ---------------------------------------------------------------------- 核函数

def cubic_kernel(r: np.ndarray) -> np.ndarray:
"""三次核 φ(r) = r³（GSDE 使用）。"""
return np.asarray(r, dtype=float) ** 3

def linear_kernel(r: np.ndarray) -> np.ndarray:
"""线性核 φ(r) = r。"""
return np.asarray(r, dtype=float)

def thin_plate_kernel(r: np.ndarray) -> np.ndarray:
"""薄板样条核 φ(r) = r² · ln r（约定 r=0 时取 0）。"""
r = np.asarray(r, dtype=float)
out = np.zeros_like(r)
m = r > _TINY
out[m] = r[m] ** 2 * np.log(r[m])
return out

def gaussian_kernel(r: np.ndarray, eps: float = 1.0) -> np.ndarray:
"""高斯核 φ(r) = exp(−(εr)²)。"""
r = np.asarray(r, dtype=float)
return np.exp(-(eps * r) ** 2)

def _pairwise_dist(A: np.ndarray, B: np.ndarray) -> np.ndarray:
"""欧氏距离矩阵 (n, m)。"""
diff = A[:, None, :] - B[None, :, :]
return np.sqrt(np.einsum("nmk,nmk->nm", diff, diff))

# ---------------------------------------------------------------------- 主类

class RBFInterpolant:
"""三次（默认）RBF + 线性多项式尾项插值器。

用法：
m = RBFInterpolant().fit(X, y) # 或 fit_rbf(X, y)
yhat = m(Xs) # 也支持 m.predict(Xs)
"""

def __init__(self,
kernel: Callable[[np.ndarray], np.ndarray] = cubic_kernel,
smooth: float = 0.0,
normalize: bool = True):
"""
kernel : 径向基核，默认三次核（GSDE 的选择）
smooth : 正则化/平滑量 λ（0 = 严格插值；样本有噪声时取小正数）
normalize : 是否把输入归一化到 [0,1]^d（强烈建议保持 True，改善条件数）
"""
self.kernel = kernel
self.smooth = float(smooth)
self.normalize = bool(normalize)
self.X: Optional[np.ndarray] = None # 归一化后的训练点
self.y: Optional[np.ndarray] = None
self.lam: Optional[np.ndarray] = None # 径向基系数
self.coef: Optional[np.ndarray] = None # 多项式尾项系数 [c0, c1, ..., cd]
self.xmin: Optional[np.ndarray] = None
self.xspan: Optional[np.ndarray] = None

# ---- 工具

def _to_unit(self, A: np.ndarray) -> np.ndarray:
A = np.asarray(A, dtype=float)
if not self.normalize:
return A
return (A - self.xmin) / self.xspan

def _design(self, X: np.ndarray) -> np.ndarray:
"""多项式尾项设计矩阵 [1, x1, ..., xd]。"""
return np.hstack([np.ones((X.shape[0], 1)), X])

# ---- 拟合

def fit(self, X: np.ndarray, y: np.ndarray) -> "RBFInterpolant":
X = np.asarray(X, dtype=float)
y = np.asarray(y, dtype=float).ravel()
if X.ndim == 1:
X = X.reshape(-1, 1)
n, d = X.shape
if n != y.size:
raise ValueError("X 与 y 的样本数不一致")
if n < d + 2:
raise ValueError(f"样本数不足：需要 ≥ d+2 = {d + 2} 个，实到 {n} 个")
self.xmin = X.min(axis=0)
```

```
self.xspan = X.max(axis=0) - self.xmin
self.xspan[self.xspan <= 0] = 1.0
Z = self._to_unit(X)
self.X, self.y = Z, y
Phi = self.kernel(_pairwise_dist(Z, Z))
if self.smooth > 0:
Phi[np.diag_indices_from(Phi)] += self.smooth
P = self._design(Z)
m = P.shape[1]
A = np.zeros((n + m, n + m))
A[:n, :n] = Phi
A[:n, n:] = P
A[n:, :n] = P.T
rhs = np.concatenate([y, np.zeros(m)])
sol = np.linalg.lstsq(A, rhs, rcond=None)[0] # 用 lstsq 而非 solve：容忍近似共线
self.lam, self.coef = sol[:n], sol[n:]
return self
# ---- 预测
def predict(self, Xs: np.ndarray) -> np.ndarray:
Xs = np.asarray(Xs, dtype=float)
if Xs.ndim == 1:
Xs = Xs.reshape(1, -1)
Z = self._to_unit(Xs)
Phi = self.kernel(_pairwise_dist(Z, self.X))
return Phi @ self.lam + self._design(Z) @ self.coef
__call__ = predict
# ---- 诊断
def r2(self, X: np.ndarray, y: np.ndarray) -> float:
y = np.asarray(y, float).ravel()
pred = self.predict(X)
ss_res = float(np.sum((y - pred) ** 2))
ss_tot = float(np.sum((y - np.mean(y)) ** 2))
if ss_tot <= _TINY:
return 1.0 if ss_res <= _TINY else 0.0
return 1.0 - ss_res / ss_tot
def fit_rbf(X: np.ndarray, y: np.ndarray, **kw) -> RBFInterpolant:
"""便捷函数：一步拟合一个默认（三次核 + 线性尾项）RBF 模型。"""
return RBFInterpolant(**kw).fit(X, y)
def leave_one_out_rmse(X: np.ndarray, y: np.ndarray, **kw) -> float:
"""留一交叉验证 RMSE：判断这套样本/这个核是否够用（成本高：会重拟合 n 次）。"""
X = np.asarray(X, float)
y = np.asarray(y, float).ravel()
errs = []
for i in range(X.shape[0]):
keep = np.ones(X.shape[0], dtype=bool)
keep[i] = False
try:
sub = RBFInterpolant(**kw).fit(X[keep], y[keep])
errs.append(float(sub.predict(X[i].reshape(1, -1))[0]) - float(y[i]))
except (ValueError, np.linalg.LinAlgError):
errs.append(float("nan"))
return float(np.sqrt(np.nanmean(np.square(errs))))
# ---------------------------------------------------------------------- 自检
def _self_test() -> None:
rng = np.random.default_rng(11)
# 1) 插值性：光滑函数上，训练点处必须精确穿过
def f(x: np.ndarray) -> np.ndarray:
x = np.atleast_2d(np.asarray(x, float))
return np.sin(2.0 * x[:, 0]) + 0.5 * x[:, 1] ** 2
X = rng.uniform(-2.0, 2.0, size=(50, 2))
y = f(X)
mdl = fit_rbf(X, y)
assert np.max(np.abs(mdl(X) - y)) < 1e-6, "RBF 未精确通过训练点"
# 2) 线性再生性：真实函数为一次多项式时应几乎零误差
w = np.array([1.5, -2.0, 0.7])
Xl = rng.uniform(-1.0, 1.0, size=(40, 2))
yl = w[0] + Xl @ w[1:]
ml = fit_rbf(Xl, yl)
Xt = rng.uniform(-1.0, 1.0, size=(200, 2))
assert np.max(np.abs(ml(Xt) - (w[0] + Xt @ w[1:]))) < 1e-6, "线性尾项未能再生线性函数"
# 3) 测试集表现
Xte = rng.uniform(-2.0, 2.0, size=(400, 2))
assert mdl.r2(Xte, f(Xte)) > 0.9, "RBF 测试集 R² 过低"
# 4) 留一误差应远小于响应量程
loo = leave_one_out_rmse(X, y)
assert loo < 0.2 * (y.max() - y.min()), f"留一 RMSE 过大：{loo}"
# 5) 平滑量 smooth 会牺牲插值性（正则化效果）
ms = fit_rbf(X, y, smooth=1e-2)
assert np.max(np.abs(ms(X) - y)) > 1e-6, "smooth>0 时不应仍精确插值"
# 6) 不同核都能跑通并给出合理精度
for kern in (cubic_kernel, linear_kernel, thin_plate_kernel,
lambda r: gaussian_kernel(r, eps=1.0)):
mk = fit_rbf(X, y, kernel=kern)
assert mk.r2(Xte, f(Xte)) > 0.5, f"核 {kern} 精度过低"
# 7) 异常输入：维数/样本数/坐标维不匹配
for bad in [lambda: fit_rbf(np.zeros((5, 2)), np.zeros(4)),
lambda: fit_rbf(np.zeros((3, 2)), np.zeros(3))]:
try:
bad()
except ValueError:
pass
```

```
else:
raise AssertionError("非法输入未报错")
try:
mdl(rng.uniform(size=(3, 5)))
except (ValueError, IndexError):
pass
else:
raise AssertionError("预测维数不匹配未报错")
print(f"[rbf.py] 自检通过：插值性 / 线性再生 / 测试R²={mdl.r2(Xte, f(Xte)):.3f} / "
f"留一RMSE={loo:.4f} / 四种核 / 异常输入")
if __name__ == "__main__":
_self_test()
```

code/sampling.py

```
"""sampling.py —— 试验设计（DoE）与尺度变换工具
本模块只依赖 numpy 与标准库，提供全书各讲算法都要用到的两件小事：
1. **拉丁超立方采样（Latin Hypercube Sampling, LHS）**
把每一维等分成 n 个区间，每维在每个区间内恰好取一个点，再随机配对。
相比纯随机采样，LHS 在样本量很少时也能保证一维投影上的均匀覆盖，
这正是昂贵黑箱优化（HEB）第一步最需要的性质。
依据：本全集的论文 07（GSDE）、论文 08（KT-ASO）、论文 11（DA-EGO）、
论文 16（TNO）、论文 18（P-ResUNet）、论文 19（SHAP）均明确使用 LHS
生成初始样本（如论文 11 用 50 个、论文 08 用 11・d_L 个）。
2. **单位立方体 <-> 物理空间的线性尺度变换**
绝大多数代理模型假设输入已归一化到 [0,1]^d，因此需要在"优化器看到的
空间"与"仿真器看到的空间"之间来回变换。
运行自检：
python code/sampling.py
自检内容：LHS 的分层性质（每维每个区间恰一个点）、边界正确性、往返变换一致性。
"""
from __future__ import annotations
from typing import Optional, Sequence, Tuple
import numpy as np
__all__ = [
"latin_hypercube",
"random_uniform",
"to_unit",
"from_unit",
"scale_matrix",
"sobol_first_order_indices",
]
def _as_bounds(bounds: Sequence[Sequence[float]]) -> np.ndarray:
b = np.asarray(bounds, dtype=float)
if b.ndim != 2 or b.shape[1] != 2:
raise ValueError("bounds 必须是形如 [(lo, hi), ...] 的 (d, 2) 数组")
if np.any(b[:, 1] <= b[:, 0]):
raise ValueError("每个维度的上界必须严格大于下界")
return b
def latin_hypercube(n: int,
dim: int,
rng: Optional[np.random.Generator] = None,
criterion: str = "random") -> np.ndarray:
"""生成 n 个 dim 维拉丁超立方样本，返回值域为 [0, 1]^dim。
参数
----
n : 样本数（必须 >= 1）
dim : 维度（必须 >= 1）
rng : numpy 随机发生器；为 None 时用 default_rng()
criterion :
"random" —— 每层内随机取点（标准 LHS）
"center" —— 每层取区间中点（即确定性分层，等价于均匀设计的近似）
返回
----
(n, dim) 数组，每行一个样本，取值在 [0, 1) 内。
实现要点
--------
先对每一维独立生成一个 0..n-1 的随机排列（决定"第 k 个样本落在第几层"），
再在层内加一个 [0,1) 的随机偏移。这样可保证：对任意一维，n 个样本
恰好落在 n 个互不重叠的等宽区间中 —— 这就是 LHS 的"分层性"。
"""
if n < 1 or dim < 1:
raise ValueError("n 与 dim 都必须 >= 1")
rng = np.random.default_rng() if rng is None else rng
u = np.empty((n, dim), dtype=float)
for i in range(dim):
perm = rng.permutation(n)
if criterion == "center":
offset = 0.5
elif criterion == "random":
offset = rng.random(n)
else:
raise ValueError("criterion 只支持 'random' 或 'center'")
u[:, i] = (perm + offset) / float(n)
return u
def random_uniform(n: int,
bounds: Sequence[Sequence[float]],
rng: Optional[np.random.Generator] = None) -> np.ndarray:
"""在给定 bounds 内生成 n 个均匀随机样本（作为 LHS 的对照基线）。"""
```

```
b =  as_bounds(bounds)
rng = np.random.default_rng() if rng is None else rng
u = rng.random((n, b.shape[0]))
return from_unit(u, b)
def to_unit(x: np.ndarray, bounds: Sequence[Sequence[float]]) -> np.ndarray:
"""把物理空间的点线性变换到 [0,1]^d。"""
b =  as_bounds(bounds)
x = np.atleast_2d(np.asarray(x, dtype=float))
if x.shape[1] != b.shape[0]:
raise ValueError("x 的维度与 bounds 不一致")
return (x - b[:, 0]) / (b[:, 1] - b[:, 0])
def from_unit(u: np.ndarray, bounds: Sequence[Sequence[float]]) -> np.ndarray:
"""把 [0,1]^d 的点线性变换回物理空间。"""
b =  as_bounds(bounds)
u = np.atleast_2d(np.asarray(u, dtype=float))
if u.shape[1] != b.shape[0]:
raise ValueError("u 的维度与 bounds 不一致")
return b[:, 0] + u * (b[:, 1] - b[:, 0])
def scale_matrix(bounds: Sequence[Sequence[float]]) -> Tuple[np.ndarray, np.ndarray]:
"""返回 (下界向量, 缩放向量)，便于手写 x = lo + u * span。"""
b =  as_bounds(bounds)
return b[:, 0].copy(), (b[:, 1] - b[:, 0]).copy()
def sobol_first_order_indices(samples: np.ndarray, values: np.ndarray) -> np.ndarray:
"""给定 (n, d) 样本与 (n,) 目标值，用"按维分组求条件均值"估计一阶敏感性。
这是一个**教学用的粗估计**（把每一维当作单一分组，直接比较组内均值波动
与总体方差），用于快速判断"哪个变量看起来更重要"。
严格的 Sobol 一阶指数需要成对（A/B/A_B^i）采样设计，本模块不提供；
需要严格版本请使用蒙特卡洛 Sobol 设计。
返回
----
(d,) 数组，每个元素是该维的"解释方差比"（越大越重要）。
"""
x = np.atleast_2d(np.asarray(samples, dtype=float))
v = np.asarray(values, dtype=float).ravel()
if x.shape[0] != v.shape[0]:
raise ValueError("samples 与 values 的样本数不一致")
n, d = x.shape
total_var = np.var(v)
if total_var <= 1e-300:
return np.zeros(d)
# 把每一维按中位数分成两组，比较组间均值差所解释的方差
idx = np.zeros(d)
for i in range(d):
med = np.median(x[:, i])
mask = x[:, i] <= med
if mask.sum() < 2 or (~mask).sum() < 2:
idx[i] = 0.0
continue
m1, m2 = v[mask].mean(), v[~mask].mean()
n1, n2 = mask.sum(), (~mask).sum()
# 组间方差（加权）
grand = v.mean()
between = (n1 * (m1 - grand) ** 2 + n2 * (m2 - grand) ** 2) / n
idx[i] = between / total_var
return np.clip(idx, 0.0, 1.0)
# ---------------------------------------------------------------------- 自检
def _self_test() -> None:
rng = np.random.default_rng(20240905)
# 1) LHS 的分层性：每维在 n 个等宽区间内各恰有一个点
n, d = 32, 6
u = latin_hypercube(n, d, rng=rng)
assert u.shape == (n, d), "LHS 形状错误"
assert (u >= 0).all() and (u < 1).all(), "LHS 越界"
for i in range(d):
bins = np.floor(u[:, i] * n).astype(int)
assert sorted(bins.tolist()) == list(range(n)), f"第 {i} 维不满足分层性"
# 2) center 模式是确定性的，且每层落在中点附近
uc = latin_hypercube(n, d, criterion="center")
assert np.allclose(uc * n - np.floor(uc * n), 0.5), "center 模式未落在层中点"
# 3) 尺度变换往返一致
bounds = [(0.0, 1.0), (-2.0, 3.5), (1.0e-3, 2.0e-3)]
up = to_unit(np.array([[0.5, 1.0, 1.5e-3]]), bounds)
back = from_unit(up, bounds)
assert np.allclose(back, [[0.5, 1.0, 1.5e-3]]), "往返变换不一致"
lo, span = scale_matrix(bounds)
# lo=[0,-2,1e-3]，span=[1,5.5,1e-3]，故 50% 处应为 [0.5, 0.75, 1.5e-3]
assert np.allclose(lo + 0.5 * span, [0.5, 0.75, 1.5e-3]), "scale_matrix 结果不符"
# 4) 敏感性粗估计：y 只依赖第 1 维时，第 1 维应显著更重要
xs = rng.random((400, 3))
ys = np.sin(3.0 * xs[:, 1]) + 0.02 * rng.standard_normal(400)
s = sobol_first_order_indices(xs, ys)
assert np.argmax(s) == 1, f"敏感性排序错误：{s}"
# 5) 错误输入应报错
for bad in ([(1.0, 0.0)], [(0.0, 1.0, 2.0)]):
try:
 as_bounds(bad)
except ValueError:
pass
else:
raise AssertionError("非法 bounds 未报错")
print("[sampling.py] 自检通过：LHS 分层性 / 边界 / 往返变换 / 敏感性排序 / 异常输入")
```

```
if __name__ == "__main__":
_self_test()
```

code/transfer.py

```
"""transfer.py —— 知识迁移：源任务筛选、负迁移检测、样本加权（论文 08 / KT-ASO）

第 07 讲（**论文 08，SW-VAE / KT-ASO**）要解决的问题是：上一个型号的优化已经产生了
成百上千个样本，能不能把它们当成"免费的知识"用到新型号上，从而少做 CFD？原文的结论是
**取得相近最优解时，KT-ASO 至少减少 50% 的函数调用**［原文 P08 摘要］。

但这条路有个致命陷阱 —— **负迁移**。原文引用 Tripp et al. 的观察：VAE 所学设计空间中的
样本分布**大致正比于其在训练集中的出现频率**。源样本只有几十个（本文每轮用 **40** 个），
而无标签叶型有上千个（本文 **1010** 个），直接混训会让源样本的特征被彻底淹没；更糟的是
源样本是"已完成任务的优化解"，往往落在训练集分布之外，重参数化回来就已经失真。

本模块把这条链路中**可计算、可验证**的几个环节实现出来：

| 函数 | 作用 | 与论文的对应 |
| --- | --- | --- |
| `mmd2` | 用 MMD² 量化源/目标样本的分布差异 | 判断源样本是否"落在分布之外" |
| `rank_source_tasks` | 按相似度给候选源任务排序 | 选哪个历史任务来迁移 |
| `negative_transfer_check` | 用验证集误差判定是否发生负迁移 | 原文反复强调的失败模式 |
| `weighted_batches` | 每 batch 固定 n_S/n_U 比例的采样器 | **SW-VAE 的核心**：论文取 n_S/n_U = 1 |
| `trailing_edge_penalty` | 叶片几何正则项 L_P | 原文公式：E[(max{0,(r_ref_min−r_min)/r_ref_min})²] |
| `vae_loss` | 加权总损失 | 原文：L = KL + λ₁ · MSE + λ₂ · L_P，λ₁=1、λ₂=5×10⁵ |
| `ffd_control_grid` | FFD 控制点网格 | 论文 08 的解码器末端 FFD 层：6×5 网格 |

**未包含的部分**：VAE 本身的网络结构与训练循环（需要深度学习框架，本代码库限定
只依赖 numpy）。"把源样本当**低保真数据**"的多保真 GP 部分，请用
`code/multifidelity.py` 的 `CoKriging`（把源任务样本作为 X_lf/y_lf、目标任务样本作为
X_hf/y_hf），这正是原文 ρ 相关结构的用法。

**只依赖 numpy**。运行自检：
python code/transfer.py
"""

from __future__ import annotations

from typing import Dict, List, Optional, Sequence, Tuple

import numpy as np

__all__ = [
"mmd2", "rank_source_tasks", "negative_transfer_check",
"weighted_batches", "trailing_edge_penalty", "vae_loss",
"ffd_control_grid",
]

_TINY = 1e-12

# ---------------------------------------------------------------------- 分布差异

def mmd2(X: np.ndarray, Y: np.ndarray, sigma: Optional[float] = None) -> float:
"""高斯核 MMD²（最大均值差异的平方），衡量两组样本的分布差异。

MMD² = E[k(x,x')] + E[k(y,y')] − 2 · E[k(x,y)], k(a,b)=exp(− ‖a−b‖²/(2σ²))

MMD² = 0 当且仅当两个分布相同（在核的特征空间意义下）；值越大差异越大。

sigma 未给定时，用两组样本合并后的**中位距离**启发式设定（常用做法）。
"""
X = np.atleast_2d(np.asarray(X, float))
Y = np.atleast_2d(np.asarray(Y, float))
if X.shape[1] != Y.shape[1]:
raise ValueError("两组样本的维度不一致")
if X.shape[0] == 0 or Y.shape[0] == 0:
raise ValueError("样本集不能为空")

def sqdist(A: np.ndarray, B: np.ndarray) -> np.ndarray:
diff = A[:, None, :] - B[None, :, :]
return np.einsum("nmk,nmk->nm", diff, diff)

if sigma is None:
allsq = np.concatenate([sqdist(X, X).ravel(), sqdist(Y, Y).ravel()])
med = float(np.median(allsq))
sigma = float(np.sqrt(med / 2.0)) if med > _TINY else 1.0
if sigma <= 0:
raise ValueError("sigma 必须为正")
g = 2.0 * sigma ** 2
kxx = float(np.mean(np.exp(-sqdist(X, X) / g)))
kyy = float(np.mean(np.exp(-sqdist(Y, Y) / g)))
kxy = float(np.mean(np.exp(-sqdist(X, Y) / g)))
return float(max(kxx + kyy - 2.0 * kxy, 0.0))

def rank_source_tasks(X_target: np.ndarray,
sources: Sequence[Tuple[str, np.ndarray]],
sigma: Optional[float] = None) -> List[Tuple[str, float]]:
"""按"与目标的分布差异"给候选源任务排序（MMD² 越小越相似，越适合迁移）。

返回 [(任务名, MMD²), ...]，按 MMD² 升序。
"""
out = [(name, mmd2(X_target, Xs, sigma=sigma)) for name, Xs in sources]
return sorted(out, key=lambda kv: kv[1])

# ---------------------------------------------------------------------- 负迁移判定

def negative_transfer_check(err_with_source: float,
err_without_source: float,
tol: float = 0.0) -> Dict[str, object]:
"""判定是否发生**负迁移**：加入源样本后，验证集误差反而变差。

返回 dict：{'negative': bool, 'delta': 误差变化量, 'ratio': 相对恶化幅度}
tol 允许一个容忍带（例如 1e-3），避免把数值噪声当成负迁移。
"""
a = float(err_with_source)
b = float(err_without_source)
delta = a - b
ratio = (delta / abs(b)) if abs(b) > _TINY else float("nan")
return {"negative": bool(delta > tol), "delta": delta, "ratio": ratio}

# ---------------------------------------------------------------------- 样本加权批次（SW-VAE）
```

```
def weighted_batches(n_source: int,
n_unlabeled: int,
n_source_per_batch: int,
ratio: float = 1.0,
n_batches: int = 10,
seed: int = 0) -> List[Tuple[np.ndarray, np.ndarray]]:
"""生成**每个 batch 内源样本数与无标签样本数比例固定**的采样方案。
这是 SW-VAE 的第一层解法［原文 P08 §第三步］：在每个 epoch 的每个 batch 中，
固定源样本数 n⁽ˢ⁾ 与无标签样本数 n⁽ᵀ⁾ 的比例（**实验中取 n⁽ˢ⁾/n⁽ᵀ⁾ = 1**），
分别从源样本集 X_S 与无标签集 X_U 中随机抽取。
这样哪怕源样本只占全集的极小一部分，它们的特征也能被稳定地学进去，
从而避免"源样本被淹没"。
返回 [(源样本下标数组, 无标签样本下标数组), ...]，共 n_batches 个 batch。
"""
if n_source <= 0 or n_unlabeled <= 0:
raise ValueError("源样本数与无标签样本数均需为正")
if n_source_per_batch <= 0 or n_source_per_batch > n_source:
raise ValueError("每批源样本数应落在 [1, n_source]")
if n_batches <= 0:
raise ValueError("batch 数量必须为正")
if ratio <= 0:
raise ValueError("比例必须为正")
n_unl_per_batch = max(1, int(round(n_source_per_batch / ratio)))
if n_unl_per_batch > n_unlabeled:
raise ValueError("按比例算出的无标签样本数超过其总量")
rng = np.random.default_rng(seed)
batches = []
for _ in range(int(n_batches)):
s = rng.choice(n_source, size=n_source_per_batch, replace=False)
u = rng.choice(n_unlabeled, size=n_unl_per_batch, replace=False)
batches.append((np.sort(s), np.sort(u)))
return batches
# ---------------------------------------------------------------------------- 几何正则项
def trailing_edge_penalty(r_min: np.ndarray, r_min_ref: float) -> float:
"""叶片几何专属正则项 L_P［原文 P08 公式］：
L_P = E[ ( max{ 0, (r_min^ref − r_min) / r_min^ref } )² ]
含义：**约束尾缘不薄于基准**，防止生成畸形叶型。
r_min 是一批叶型的尾缘最小厚度（或半径），r_min^ref 是基准值。
"""
r = np.asarray(r_min, float).ravel()
if r.size == 0:
raise ValueError("r_min 不能为空")
if abs(float(r_min_ref)) <= _TINY:
raise ValueError("基准尾缘厚度 r_min^ref 必须非零")
viol = np.maximum(0.0, (float(r_min_ref) - r) / float(r_min_ref))
return float(np.mean(viol ** 2))
def vae_loss(kl: float, mse: float, lp: float,
lam1: float = 1.0, lam2: float = 500000.0) -> float:
"""SW-VAE 的总损失［原文 P08 公式］：L = KL + λ₁ · MSE + λ₂ · L_P。
实验取值 **λ₁ = 1、λ₂ = 500000**（原文明确给出）。
"""
return float(kl) + float(lam1) * float(mse) + float(lam2) * float(lp)
# ---------------------------------------------------------------------------- FFD 控制网格
def ffd_control_grid(nu: int = 6, nv: int = 5) -> Tuple[np.ndarray, int]:
"""生成 FFD（自由变形）控制点网格的坐标与总数。
论文 08 在解码器末端嵌入 FFD 层，**控制点网格为 6×5，主动变量共 50 个**
［原文 Table 3、§5.2］，用以保证生成叶型的光顺性。
返回 (坐标数组 (nu · nv, 2), 控制点总数)。
"""
if nu < 2 or nv < 2:
raise ValueError("网格至少为 2×2")
u = np.linspace(0.0, 1.0, int(nu))
v = np.linspace(0.0, 1.0, int(nv))
U, V = np.meshgrid(u, v, indexing="ij")
return np.column_stack([U.ravel(), V.ravel()]), int(nu) * int(nv)
# ---------------------------------------------------------------------------- 自检
def _self_test() -> None:
rng = np.random.default_rng(20240905)
# ---- 1) MMD²
X = rng.normal(0.0, 1.0, size=(200, 3))
Y = rng.normal(0.0, 1.0, size=(200, 3))
Z = rng.normal(6.0, 1.0, size=(200, 3))
assert abs(mmd2(X, X)) < 1e-12, "同分布样本与自身的 MMD² 应为 0"
assert abs(mmd2(X, Y) - mmd2(Y, X)) < 1e-12, "MMD² 应对称"
assert mmd2(X, Y) < mmd2(X, Z), "分布差异更大的一组 MMD² 应更大"
assert mmd2(X, Y) >= 0.0, "MMD² 不应为负"
# ---- 2) 源任务排序：越相似的排得越前
T = rng.normal(0.0, 1.0, size=(100, 2))
S_near = rng.normal(0.2, 1.0, size=(100, 2))
S_far = rng.normal(5.0, 1.0, size=(100, 2))
ranked = rank_source_tasks(T, [("far", S_far), ("near", S_near)])
assert [n for n, _ in ranked] == ["near", "far"], f"源任务排序错误：{ranked}"
# ---- 3) 负迁移判定
r = negative_transfer_check(0.30, 0.20)
assert r["negative"] and abs(r["delta"] - 0.10) < 1e-12, "负迁移判定错误"
assert abs(r["ratio"] - 0.5) < 1e-12, "相对恶化幅度计算错误"
r2 = negative_transfer_check(0.15, 0.20)
assert not r2["negative"], "误差下降不应判为负迁移"
assert negative_transfer_check(0.2005, 0.20, tol=1e-3)["negative"] is False, \
"容忍带内的波动不应判为负迁移"
```

```
# ---- 4) SW-VAE 的加权批次采样：每个 batch 内源/无标签比例固定
batches = weighted_batches(n_source=40, n_unlabeled=1010,
n_source_per_batch=8, ratio=1.0,
n_batches=12, seed=1)
assert len(batches) == 12, "batch 数量不对"
for s_idx, u_idx in batches:
assert s_idx.size == 8, "每个 batch 的源样本数不固定"
assert u_idx.size == 8, "ratio=1 时无标签样本数应与源样本数相同"
assert np.all(s_idx < 40) and np.all(u_idx < 1010), "下标越界"
assert len(set(s_idx.tolist())) == s_idx.size, "batch 内源样本重复"
# 比例随 ratio 变化
b2 = weighted_batches(40, 1010, 8, ratio=0.5, n_batches=3, seed=1)
assert b2[0][1].size == 16, "ratio=0.5 时无标签样本数应为源样本的 2 倍"
# 源样本被"轮转"使用：多个 batch 合起来应覆盖到大部分源样本
used = set(np.concatenate([b[0] for b in batches]).tolist())
assert len(used) >= 30, f"源样本覆盖率偏低：{len(used)}/40"
# ---- 5) 尾缘正则项 L_P
assert trailing_edge_penalty(np.array([1.0, 1.2]), r_min_ref=1.0) == 0.0, \
"不薄于基准时惩罚应为 0"
p = trailing_edge_penalty(np.array([0.5]), r_min_ref=1.0)
assert abs(p - 0.25) < 1e-12, f"半厚时的惩罚应为 0.25，实为 {p}"
p2 = trailing_edge_penalty(np.array([0.0, 1.0]), r_min_ref=1.0)
assert abs(p2 - 0.5) < 1e-12, f"期望 0.5，实为 {p2}"
# 越薄惩罚越大（单调）
vals = [trailing_edge_penalty(np.array([t]), 1.0) for t in (1.0, 0.8, 0.6, 0.4)]
assert all(vals[i] < vals[i + 1] for i in range(len(vals) - 1)), "L_P 应随厚度减小而增大"
# ---- 6) VAE 总损失：λ₁=1、λ₂=5×10⁵（原文取值）
assert abs(vae_loss(0.0, 0.0, 0.0)) < 1e-12
assert abs(vae_loss(1.0, 2.0, 3.0) - (1.0 + 2.0 + 1.5e6)) < 1e-6, "损失加权错误"
assert vae_loss(0.0, 0.0, 1e-4) > vae_loss(1.0, 1.0, 0.0), \
"尾缘惩罚被 λ₂ 放大后应主导损失（这正是原文取 λ₂=5e5 的用意）"
# ---- 7) FFD 控制点网格：论文 08 为 6×5
grid, n = ffd_control_grid(6, 5)
assert grid.shape == (30, 2) and n == 30, f"6×5 网格应有 30 个控制点，实为 {n}"
assert abs(grid[0, 0]) < 1e-12 and abs(grid[-1, 0] - 1.0) < 1e-12, "网格端点不对"
# ---- 8) 异常输入
for bad in [lambda: mmd2(np.zeros((3, 2)), np.zeros((3, 3))),
lambda: mmd2(np.zeros((0, 2)), np.zeros((3, 2))),
lambda: weighted_batches(0, 10, 2),
lambda: weighted_batches(5, 10, 9),
lambda: weighted_batches(5, 10, 2, ratio=0.0),
lambda: weighted_batches(5, 10, 2, n_batches=0),
lambda: weighted_batches(5, 3, 2, ratio=0.1),
lambda: trailing_edge_penalty(np.array([1.0]), 0.0),
lambda: trailing_edge_penalty(np.zeros(0), 1.0),
lambda: ffd_control_grid(1, 5)]:
try:
bad()
except ValueError:
pass
else:
raise AssertionError("非法输入未报错")
print(f"[transfer.py] 自检通过：MMD²(近 {mmd2(T, S_near):.4f}) < 远 {mmd2(T, S_far):.4f}) / "
f"源任务排序 / 负迁移判定 / 加权批次(8+8) / L_P / 损失加权 / FFD 6×5={n}")
if __name__ == "__main__":
_self_test()
```